Machine Learning

Lecture 1: Classical Machine Learning Fundamentals

國立臺灣大學
National Taiwan University

National Taiwan University

Department of Computer Science and Information Engineering

Prof. Chun-Yi Lee

Outline

- **Class information**
- What is Machine Learning (ML)
- ML Problem class
- Assumptions
- Evaluation / Verification
- ML types and procedures
- Basic algorithms

Class Information

- **Lecture Hours** : 16:20-19:10 (Every Fridays)
- **Location**: CSIE 104、103 (not always available)、101
- **TA Email**: ml-elsalab@googlegroups.com
- **Course Web.** : NTU Cool
- **Reference Materials** :
  - Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press
  - Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer
  - Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press
- **Requirements**:
  - Computer Programming
  - Linear Algebra
  - Calculus and Probability

Class Information

- **Grading Policy**
  - Assignments (**80%**)
    - Everyone has to submit his/her own homework
  - Final project and presentation (**20%**)
    - Please form groups of **three to four** people
- **Submission Policy**
  - You have **two** days to be used for “late submissions”
  - No copy or cheating, otherwise you will get zero points
- **Computational Resources**
  - You are responsible for your own computing resources

Final Project Specification

- **Scope**
  - ML-focused topics with novel contributions (no existing work replication)
  - Must demonstrate clear ML methodology and innovation
- **Breakdown**
  - Author Score (40%)
    - Your project quality based on peer reviews and rebuttals
  - Reviewer Score (30%)
    - Your review quality evaluated by TAs/instructor
  - Instructor Score (30%)
    - Technical assessment by TAs/instructor
  - 10 minutes presentation (30%)

Submission deadline	12/1/2025, 23:59 PM
Review deadline	12/5/2025, 23:59 PM
Rebuttal deadline	12/7/2025, 23:59 PM
Discussion deadline	12/10/2025, 23:59 PM
Decision notification	12/12/2025, 23:59 PM

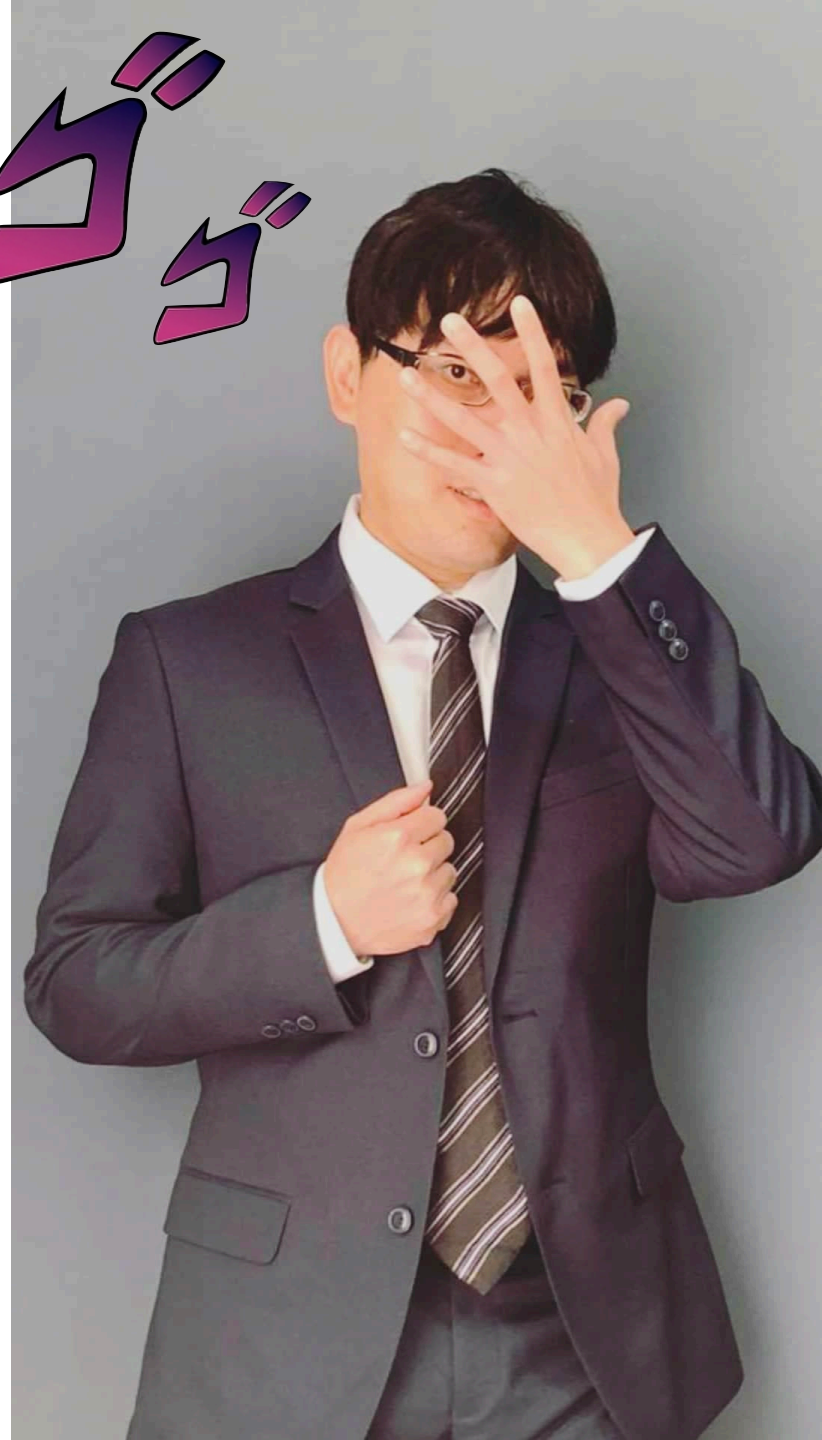
12/19 in class (Top **n**% of the submissions) (**n**: TBD)

- **Submission format:**
  - The final report has to be submitted in NeurIPS 2025 template, in English
  - 4~8 pages, using LaTeX (LaTeX source files should be submitted)
  - Clearly specify your motivation, novelty, and main contributions
  - The source codes. The results should be reproducible
    - Following this: https://github.com/paperswithcode/releasing-research-code

Your Instructor

- Prof. Chun-Yi Lee (李濬屹)
- cylee@csie.ntu.edu.tw
- Office: CS Building R510

- Office Hours :
  - Appointments by email



Course Enrollment Information

This course is specifically designed and developed for full-time enrolled Computer Science and Information Engineering students at National Taiwan University

Enrollment Policy

Please scan the QR code on the righthand side

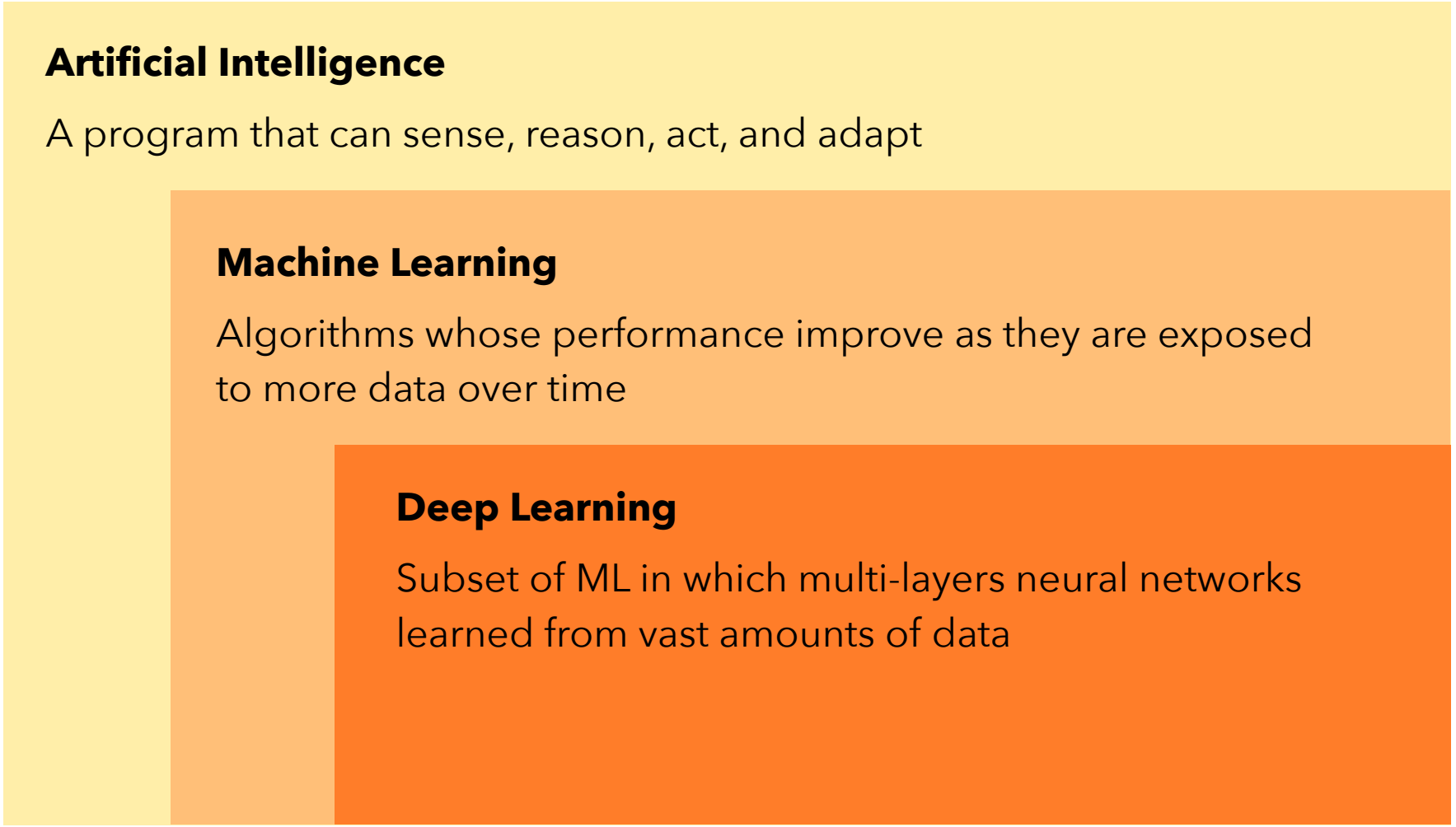
We will prioritize EECS students

Outline

- Class information
- **What is Machine Learning (ML)**
- ML Problem class
- Assumptions
- Evaluation / Verification
- ML types and procedures
- Basic algorithms

What is Machine Learning

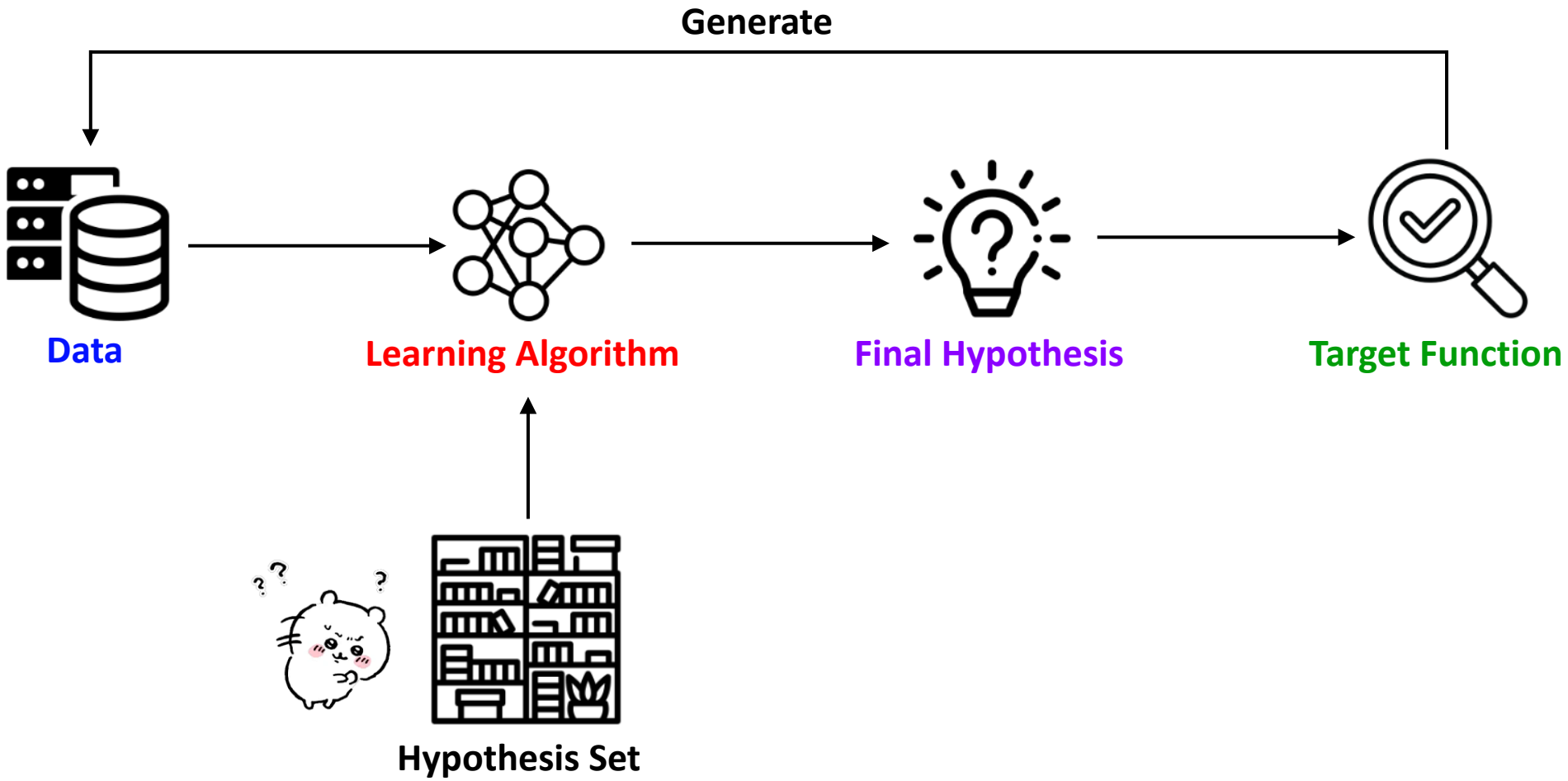
Scope of Machine Learning



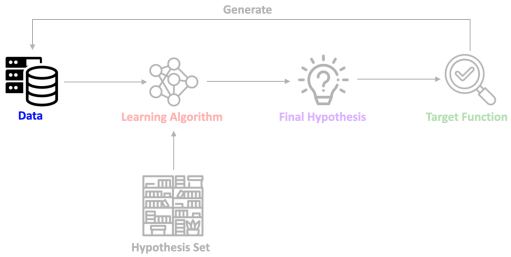
What is Machine Learning

Definition of Machine Learning

- Use data to compute hypothesis that approximates target function through learning algorithms



What is Machine Learning



Data

Data sources

- Websites
- IoT
- Databases
- Business systems
- Paper documents

What to collect

Where to collect

How to collect

Data collection methods

- Application Programming Interface
- Optical Character Recognition
- Robotic Process Automation
- Intelligent Document Processing
- Web Scraping

How much to collect

Where to store the collected data?

Data repositories

- SQL/NoSQL databases
- Data warehouse
- Data lake

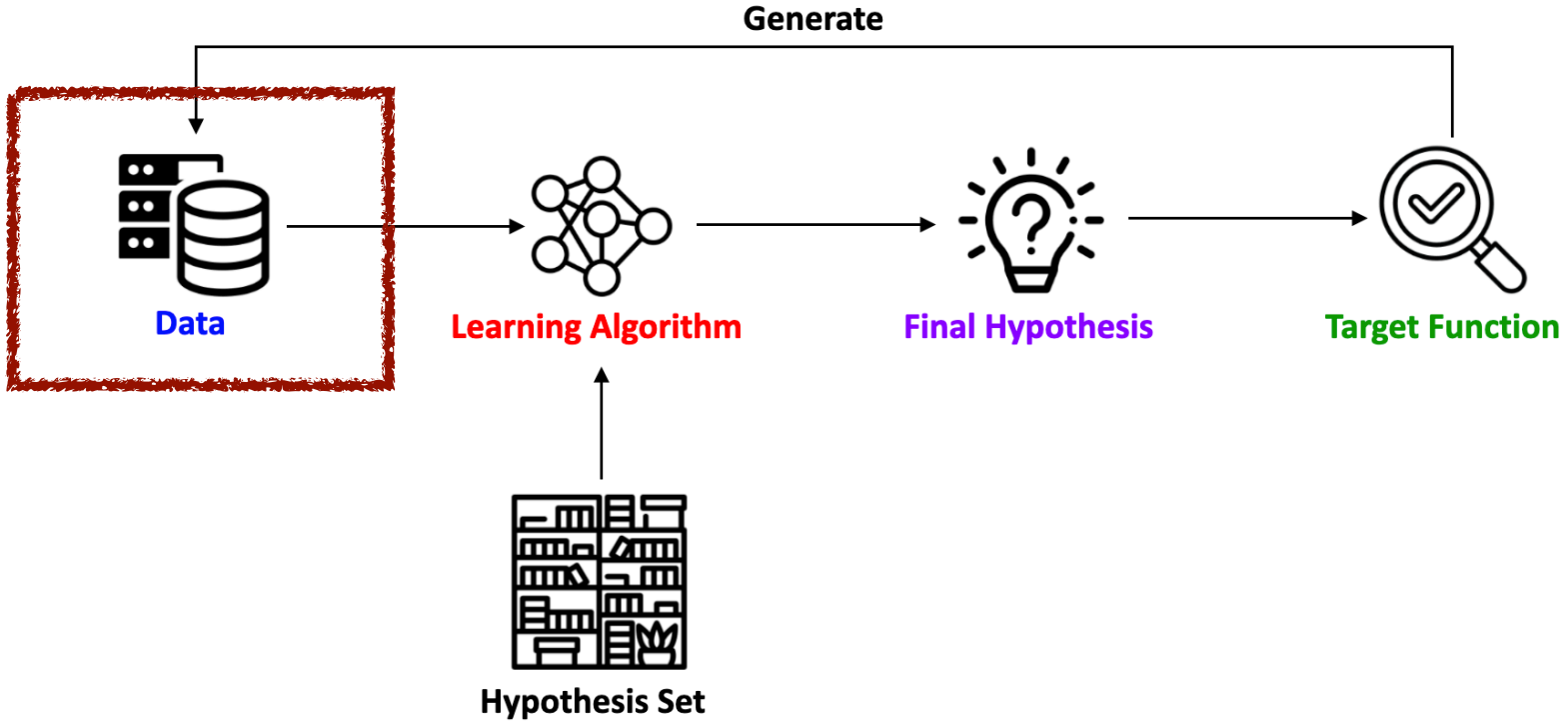
Outline

- Class information
- What is Machine Learning (ML)
- **ML Problem class**
- Assumptions
- Evaluation / Verification
- ML types and procedures
- Basic algorithms

Machine Learning Problem Class

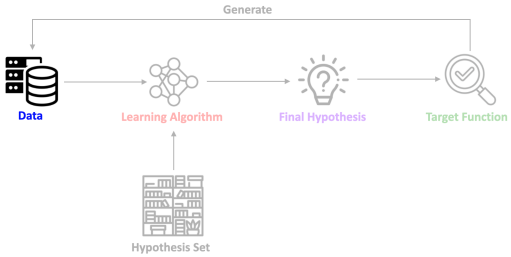
Outline

- Supervised learning
- Unsupervised learning
- Semi-supervised learning
- Self-supervised learning
- Weakly-supervised learning
- Reinforcement learning



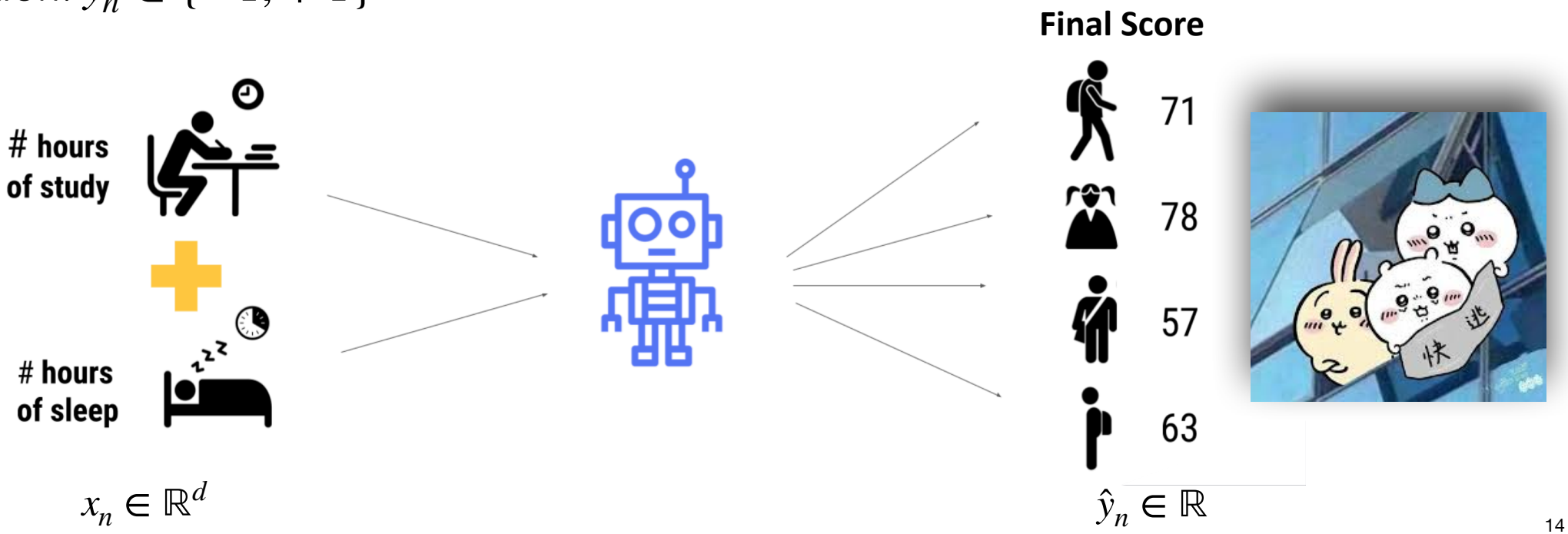
$$\mathscr{D} : \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$$

Machine Learning Problem Class



Supervised Learning

- Learning with labeled data (every x_n comes with a corresponding label y_n)
- Goal: Find a mapping from input x_n to output $\hat{y}_n$ using labeled data y_n
- Example:
  - Regression: $y_n \in \mathbb{R}$
  - Classification: $y_n \in \{-1, +1\}$



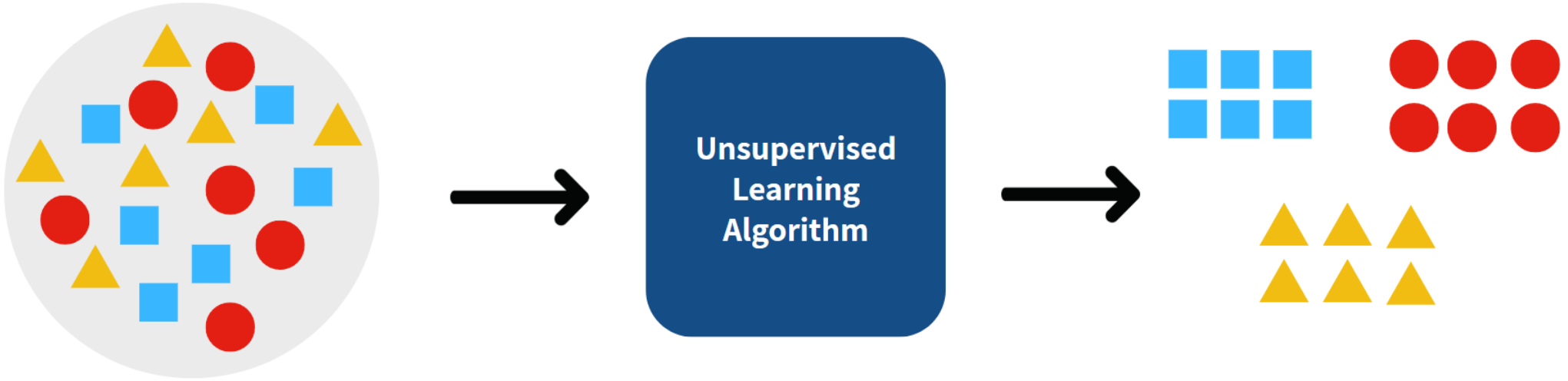
$x_n \in \mathbb{R}^d$

$\hat{y}_n \in \mathbb{R}$

Machine Learning Problem Class

Unsupervised Learning

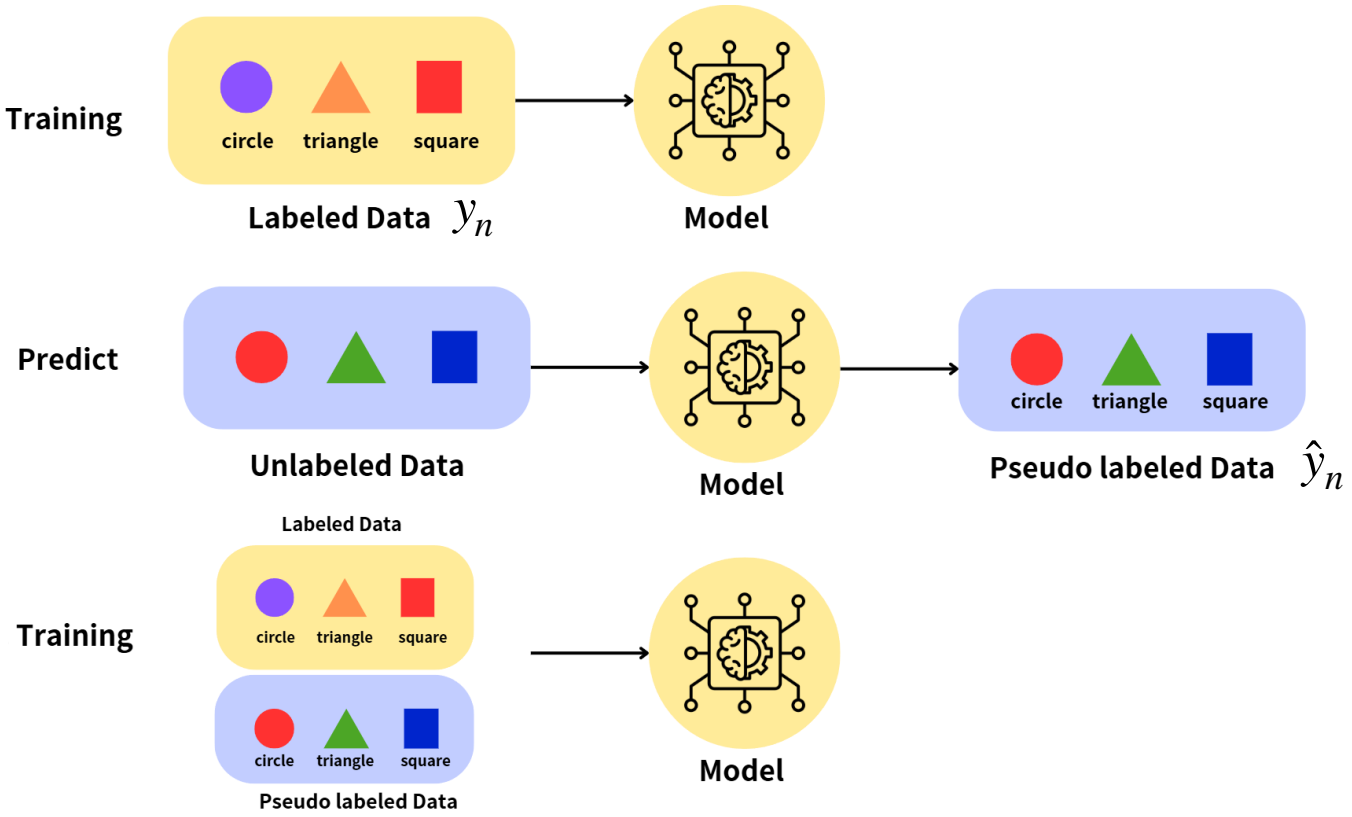
- Learning without labeled data (w/o y_n)
- **Goal**: Find hidden patterns or structures in data without labels
- Example:
  - Clustering: Group similar data points into clusters
  - Density estimation: Estimate underlying distribution of data by figuring out how likely different values of regions are



Machine Learning Problem Class

Semi-supervised Learning

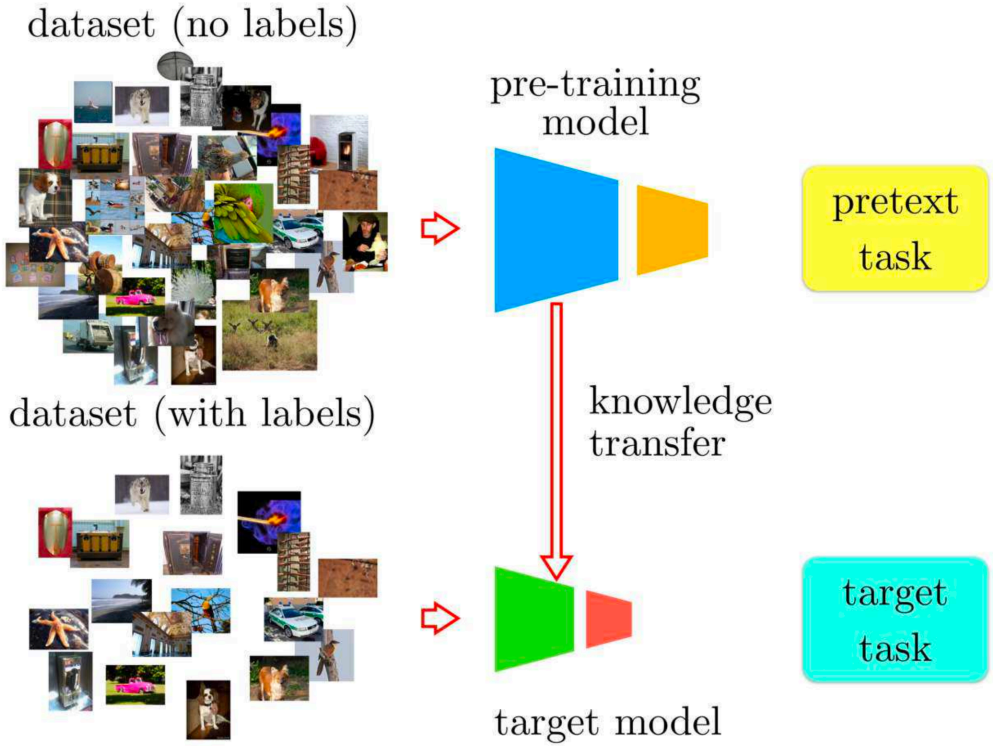
- Learning from a small amount of labeled data combined with a large amount of unlabeled data
- Goal: Improve performance while reducing the need for labeled data



Machine Learning Problem Class

Self-supervised Learning

- Design a pretext task to learn pseudo label from unlabeled data
- Goal: Solve pretext task, forcing the model to learn meaningful structure, or patterns in the data that later help with downstream/target tasks (i.e. classification, detection, …)

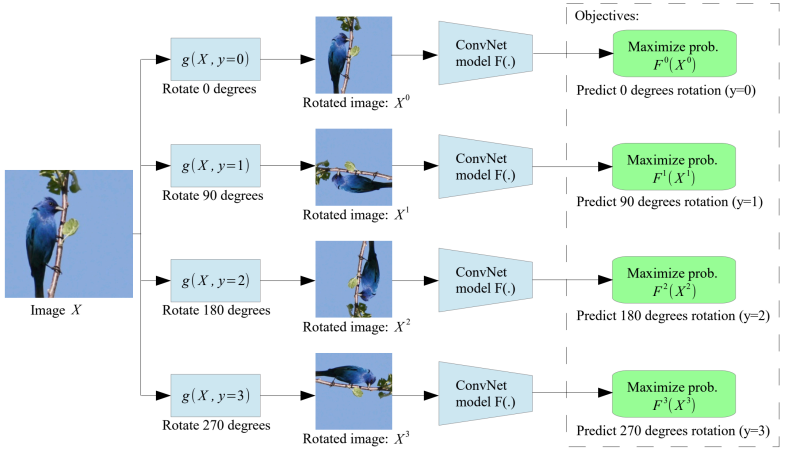


Mehdi Noroozi, et. al., Boosting Self-Supervised Learning via Knowledge Transfer, CVPR 2018.

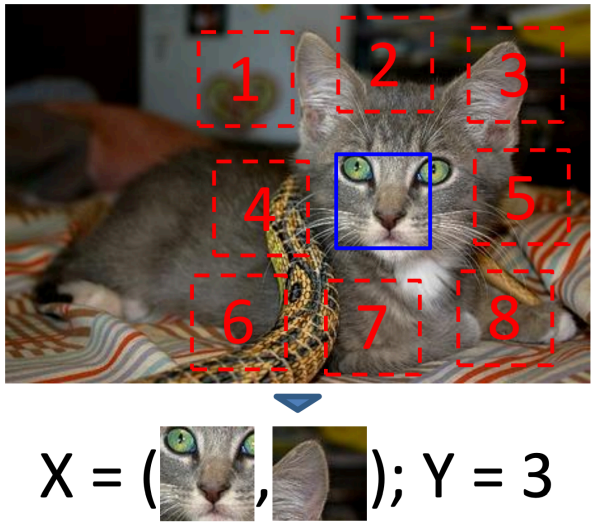
Machine Learning Problem Class

Self-supervised Learning - Predictive Methods

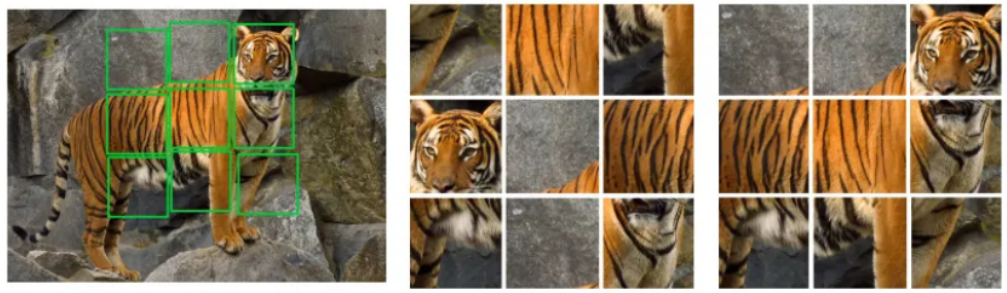
- Learn by predicting transformations of the input rather than reconstructing input



Predict images rotation (ICLR 2018)



Context Prediction (ICCV 2015)

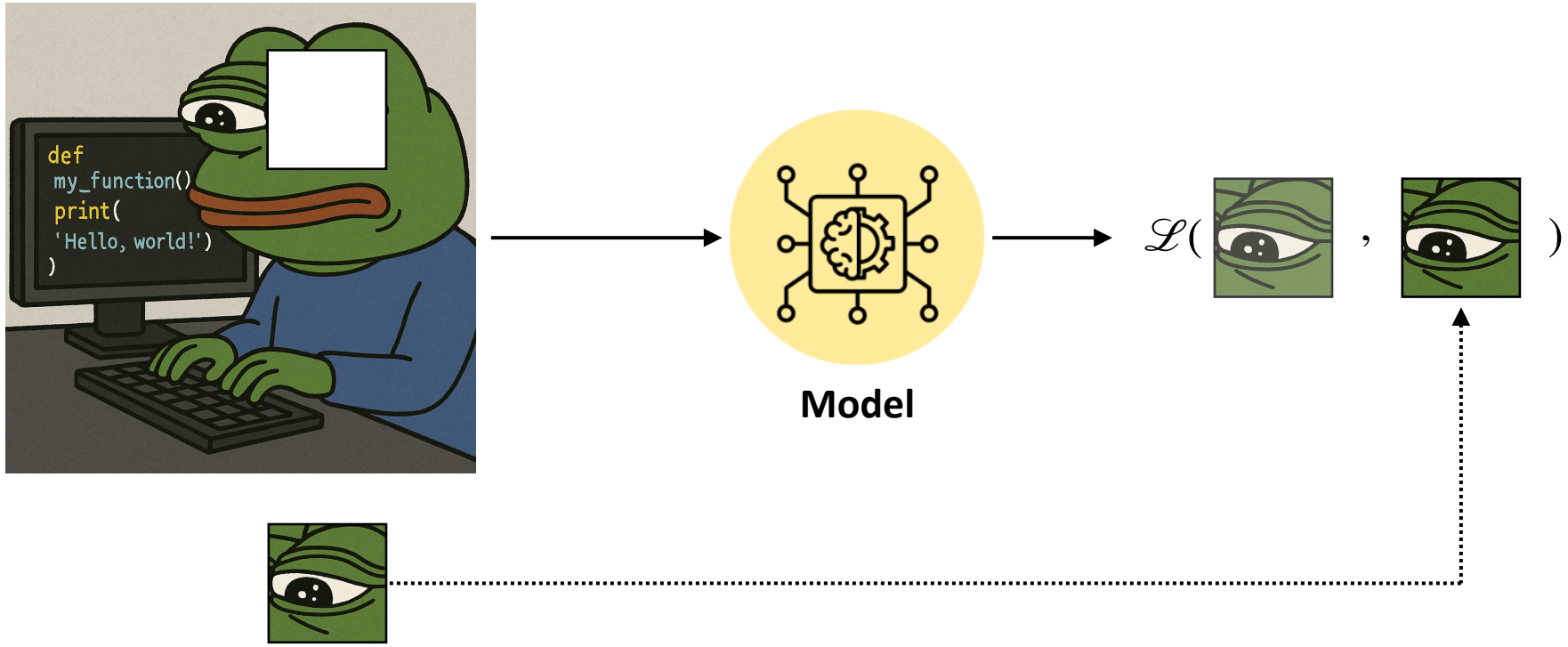


Jigsaw puzzles (ECCV 2016)

Machine Learning Problem Class

Self-supervised Learning - Generative Methods

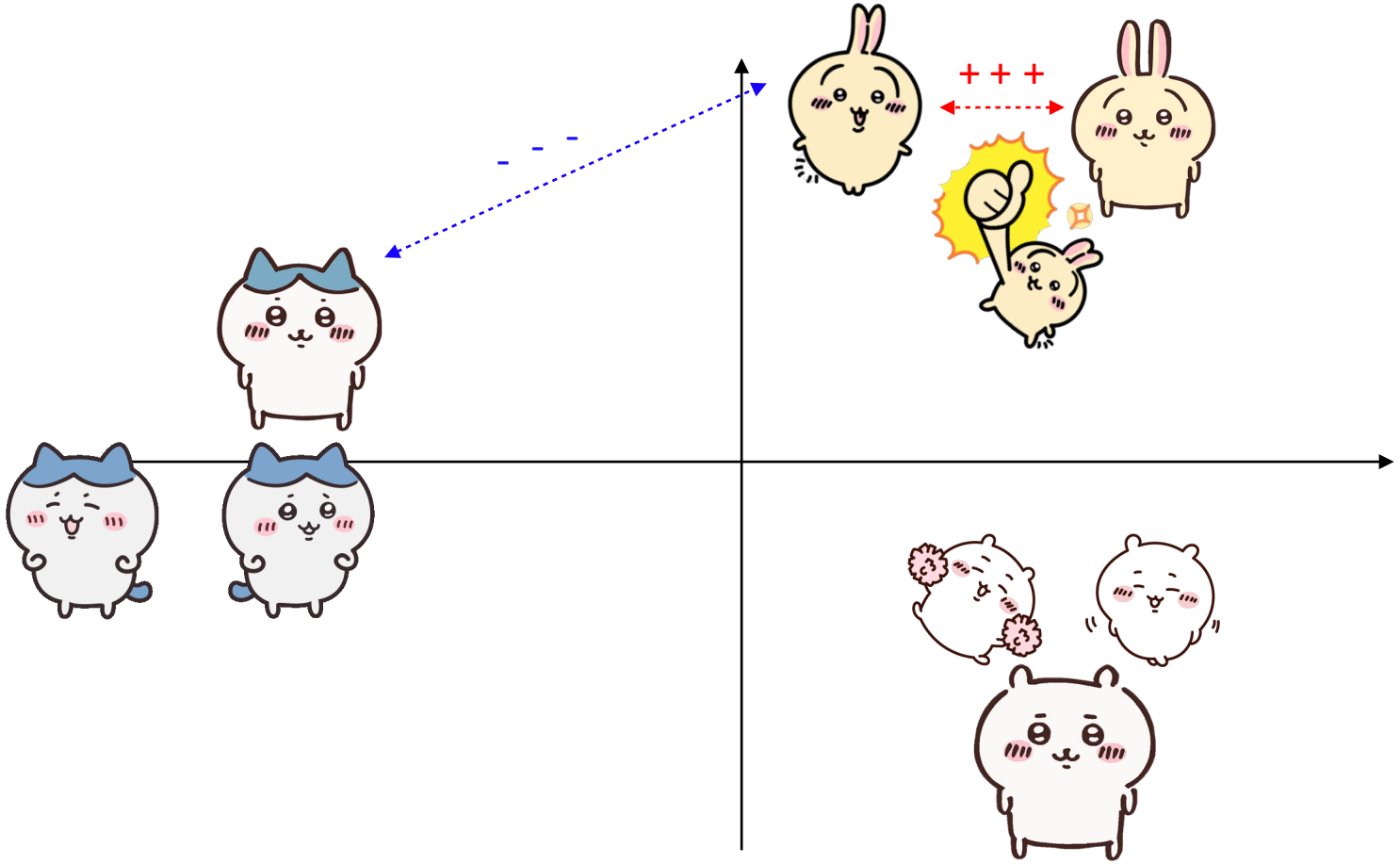
- Pixel-level reconstruction: Train the model to recover the original input from partially observed data



Machine Learning Problem Class

Self-supervised Learning - Contrastive Methods

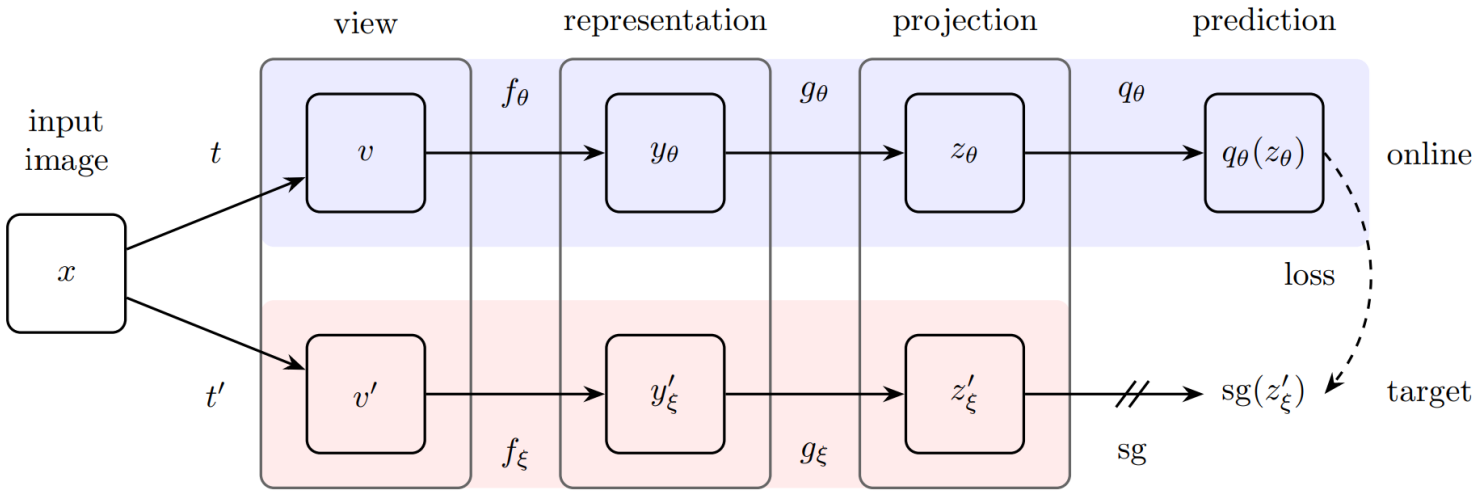
- Pull similar (positive) samples closer and push dissimilar (negative) samples farther apart



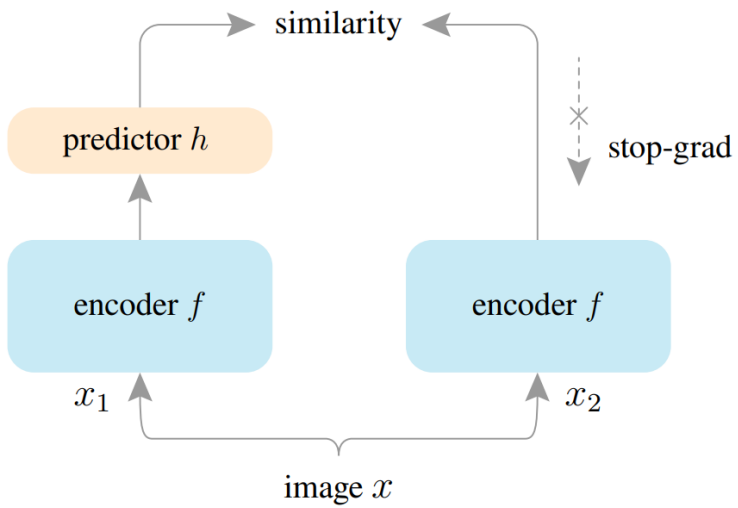
Machine Learning Problem Class

Self-supervised Learning - Bootstrapping methods

- Avoid relying on contrastive learning or negative samples. Instead, the model improves its representations by bootstrapping from its own predictions, essentially learning from itself in a teacher–student setup.



BYOL (Bootstrap your own latent)



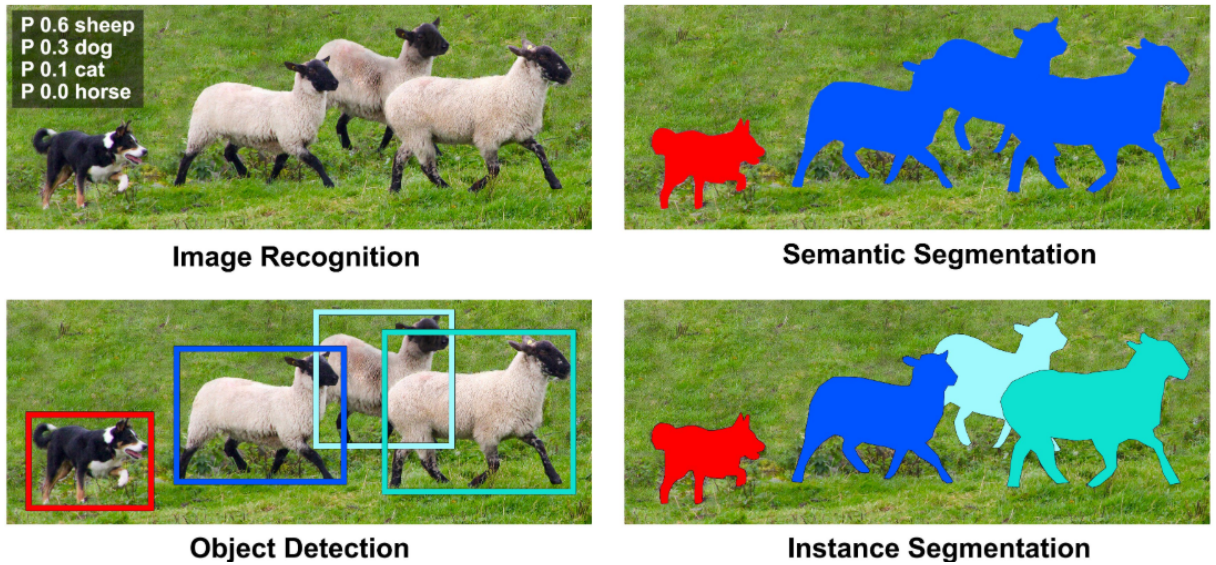
SimSiam (Simple Siamese)

CVPR 2021

Machine Learning Problem Class

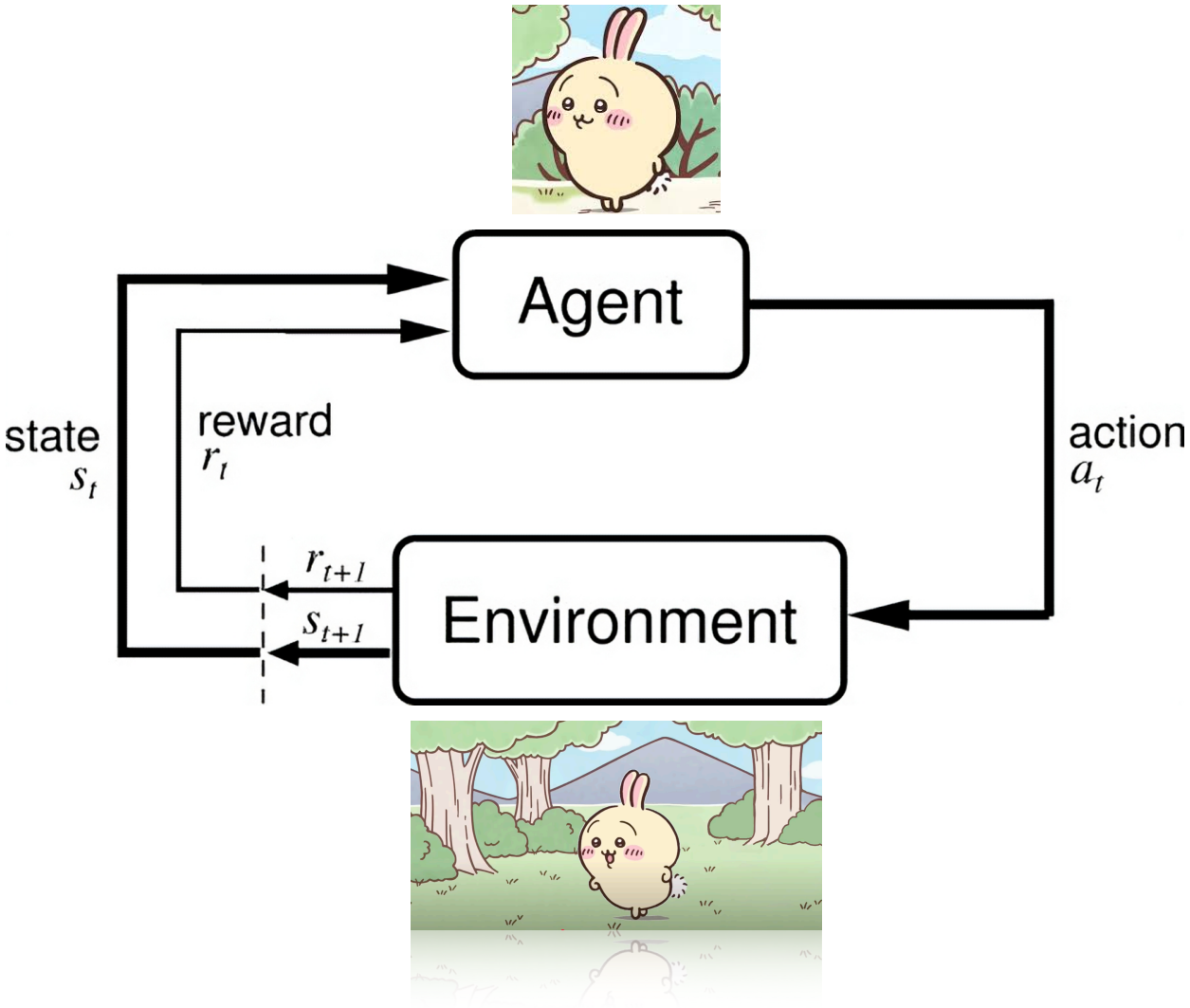
Weakly-supervised Learning

- Inexact supervision
  - Labels are coarse-grained
  - Example: A chest X-ray is labeled as “pneumonia,” but the exact abnormal area is unknown
- Inaccurate Supervision
  - Labels contain errors or noise (common in large human-labeled datasets)



Reinforcement Learning

Learn to make sequential decisions by interacting with an environment and receiving rewards or penalties



The agent

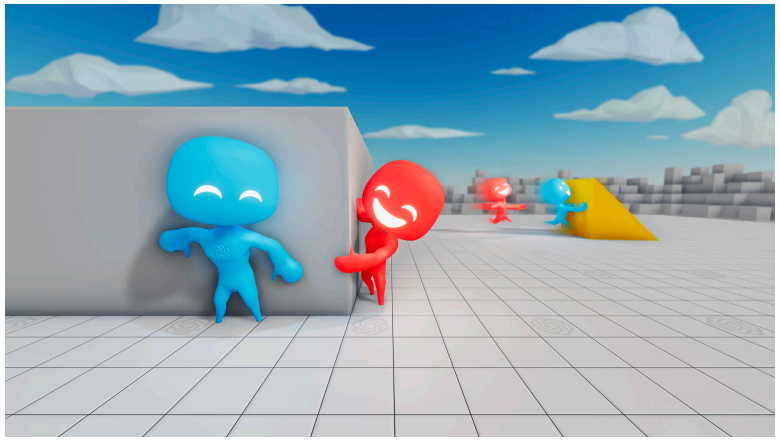
- Receives a state $s_t \in S$ (State Space)
- Outputs an action $a_t \in A$ (Action Space)
- Receives a reward $r_t \in R$ (Reward Function)

The environment

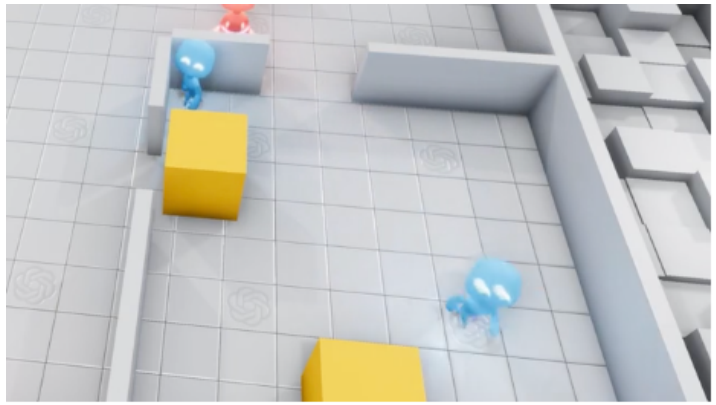
- Receives an action a_t
- Outputs the next state s_{t+1}
- Outputs a reward $r_t \in R$

Reinforcement Learning

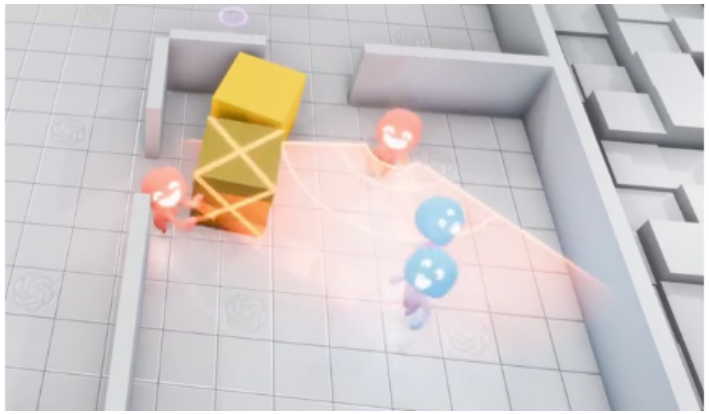
Multi-Agent Task: Hide and Seek



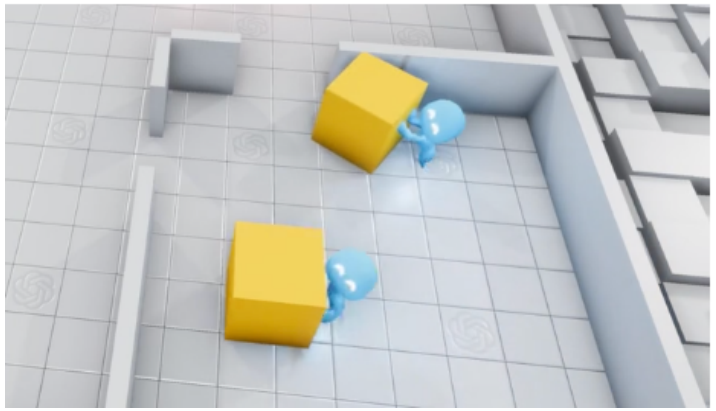
Move



Search

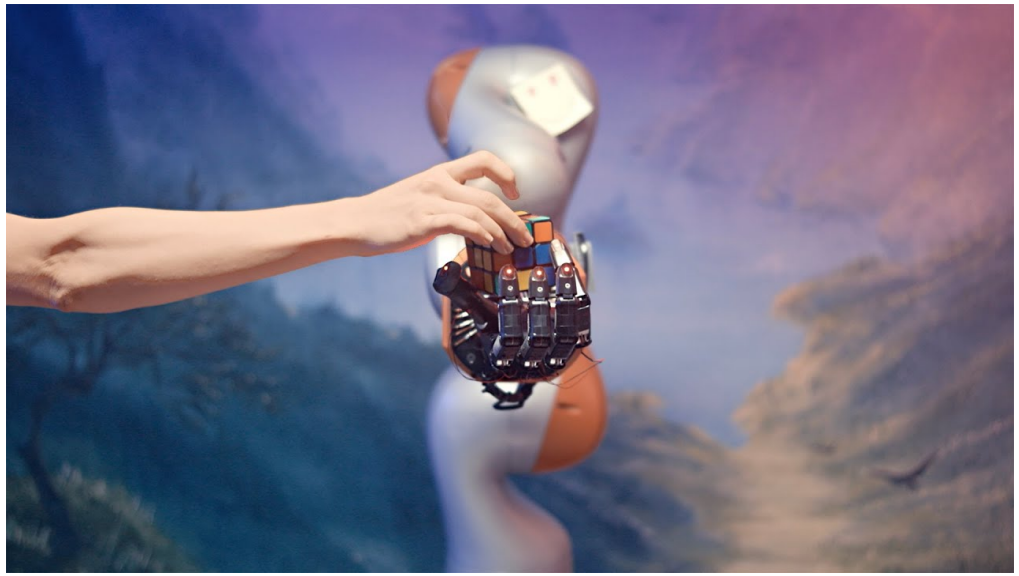


Grab



Reinforcement Learning

Robotic Arm and Hand



Outline

- Class information
- What is Machine Learning (ML)
- ML Problem class
- **Assumptions**
- Evaluation / Verification
- ML types and procedures
- Basic algorithms

Assumption of Machine Learning

Reduce the "size" or "expressiveness" of the space of possible hypotheses

- Machine learning faces a fundamental challenge: the **hypothesis space (all possible models or solutions)** is enormous. Without some way to constrain this space, learning becomes impossible or ineffective.
- **No Free Lunch Theorem**

  Without any assumptions on the learning problem, all algorithms perform equally well when **averaged over all possible problems** → no algorithm is universally better
- **Hypothesis Space is Too Large**

  The set of all possible hypotheses is enormous. Without assumptions, models may overfit to noise instead of learning the true signal and incur **computational intractability**
- **Role of Assumptions (Inductive Bias)**

  1. Assumptions guide learning toward plausible solutions
  2. Reduce hypothesis space, improving generalization ability. Help models **converge faster** and **more efficiently**.

Types of Assumption in Machine Learning

Data are independent and identically distributed (IID)

- **Identically distributed**: All data points come from the **same underlying probability distribution**
- **Independent**: Each data point is collected **separately** and does not affect others
  - For example : coin tossing, survey responses, random image classification

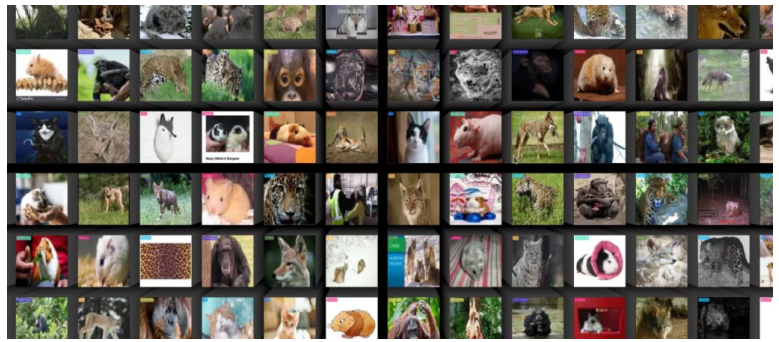
Why IID Assumption Matters ?

- **Statistical estimation**: Ensures parameter estimates (e.g., MLE) are unbiased and reliable
- **Training vs. testing**: Makes training and test sets **comparable**, supporting fair evaluation
- **Generalization**: Helps models perform well on unseen data by making training data representative

Non-IID data

- Time series data
- Imbalanced data

IID



Non-IID

Types of Assumption in Machine Learning

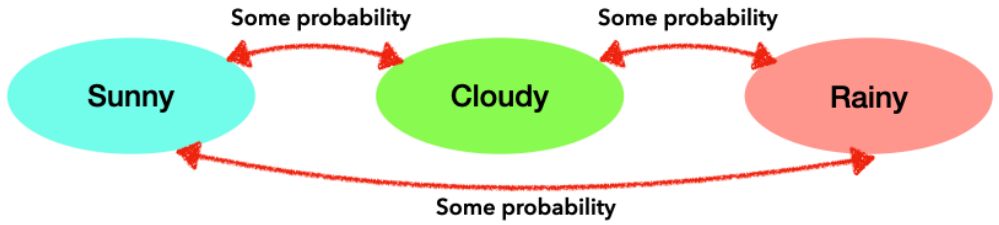
Data are generated by a Markov Chain

Markov Chain

- A Markov Chain is a mathematical model for systems that change over time, where the future depends only on the present state, not on the entire history.

$$p(s' \mid s) = P(s_{t+1} = s' \mid s_t = s)$$

- The future depends only on the present, not on how we got here
- A Markov Process is a stochastic process that satisfies the ***Markov property***
  - *Sometimes characterized as memorylessness*
  - It is a process for which predictions can be made regarding future outcomes based solely on its present state
  - Such predictions are just as good as the ones that could be made knowing the process's full history



Feature Representation

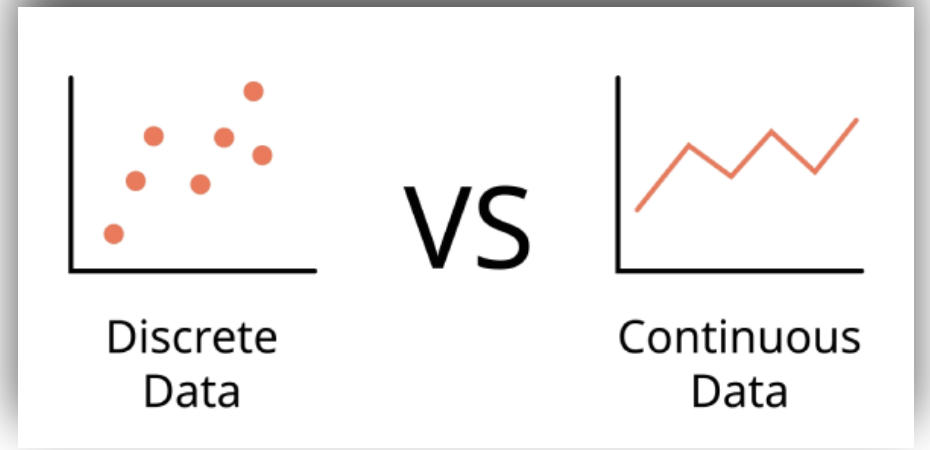
How different types of data or features are formulated?

Feature Representation

- The way raw data is **encoded into numerical form**
- Determines how models **interpret and learn** from data

Example

- **Discrete features** → categorical values (e.g., text tokens, colors, IDs)
- **Continuous features** → numeric values (e.g., height, weight, temperature)
- **Ordinal features** → ordered categories (e.g., low, medium, high)
- **Structured features** → sequences, graphs, images, etc.



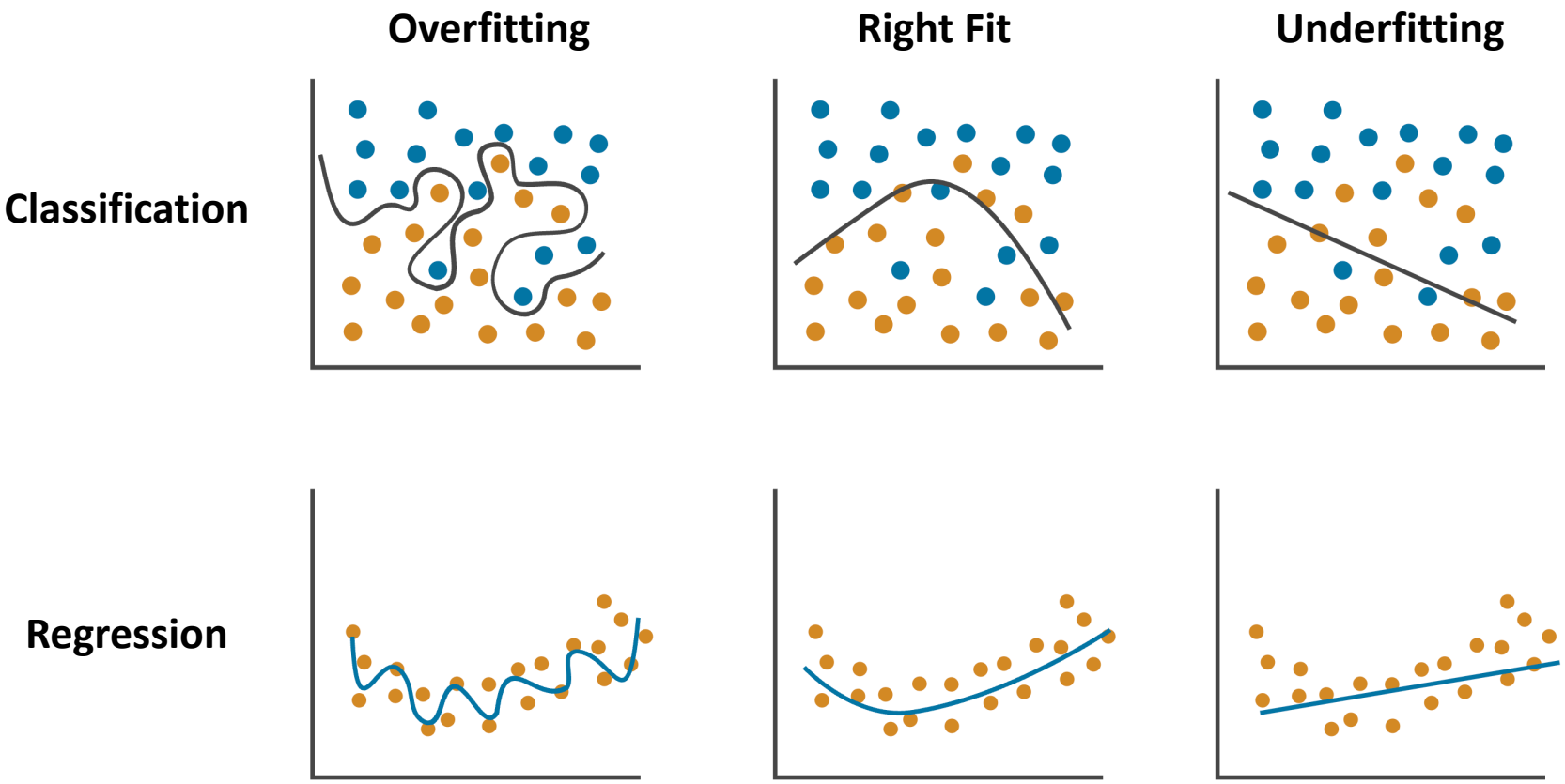
Outline

- Class information
- What is Machine Learning (ML)
- ML Problem class
- Assumptions
- **ML Evaluation / Verification**
- ML types and procedures
- Basic algorithms

Machine Learning Evaluation

Using different evaluation metrics to understand a machine learning model's performance

How does a "good" prediction look like?

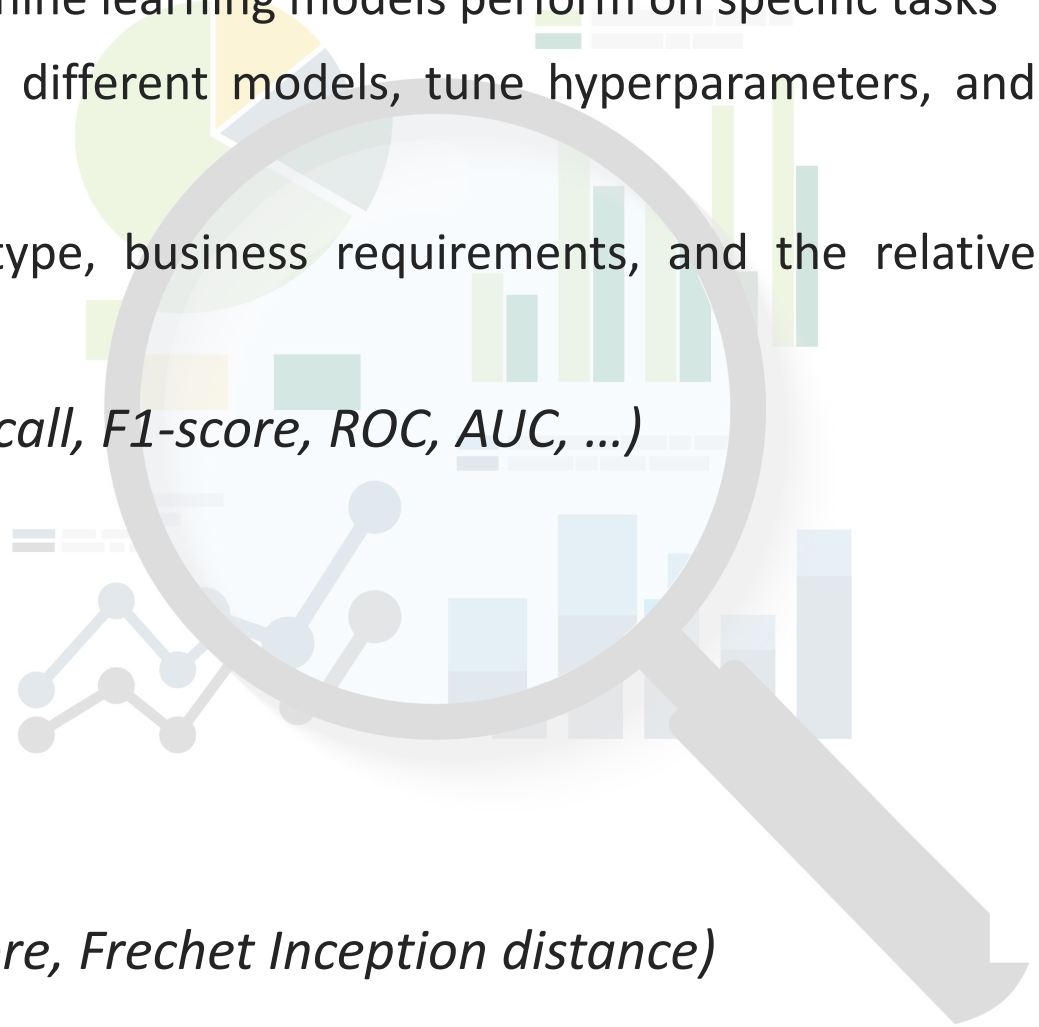


https://www.mathworks.com/discovery/overfitting.html

Machine Learning Evaluation Metrics

Comprehensive guide to measuring model performance across different tasks

- Quantitative measures used to assess how well machine learning models perform on specific tasks
- These metrics provide objective ways to compare different models, tune hyperparameters, and determine if a model is ready for deployment
- The choice of metric depends on the problem type, business requirements, and the relative importance of different types of errors
  - *Classification Metrics (accuracy, precision, recall, F1-score, ROC, AUC, …)*
  - *Regression Metrics (MSE, MAE, R2)*
  - *Ranking Metrics (MRR, DCG, NDCG)*
  - *Statistical Metrics (Correlation)*
  - *Computer Vision Metrics (PSNR, SSIM, IoU)*
  - *NLP Metrics (Perplexity, BLEU score)*
  - *Deep Learning Related Metrics (Inception score, Frechet Inception distance)*



Classification Performance Evaluation

Metrics to assess how accurately models predict categorical outcomes using confusion matrix analysis

Confusion matrix

	Actual Yes	**Actual** No
Predict Yes	TP	FP
Predict No	FN	TN

- Accuracy = (TP+TN) / Total samples
- Precision = TP / (TP+FP)
- Recall = TP / (TP+FN)
- F1 Score = $\frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$

💡 These metrics specifically evaluate categorical prediction accuracy and are derived from the confusion matrix, which is the foundation for understanding classification performance.

Evaluation of Machine Learning

Loss Function: Mathematical functions that quantify the difference between predicted and actual values

Why loss function is crucial ?

- Performance measurement
- Direction of improvement
- Balancing bias and variance
- Influencing model behavior

Different **example** loss functions

- y_i: True label/target value for sample i
- $\hat{y}_i$: Predicted value/score for sample i
- t: Threshold parameter
- $\ell(y_i, \hat{y}_i)$: Loss function measuring predictions errors

- **0-1 loss**

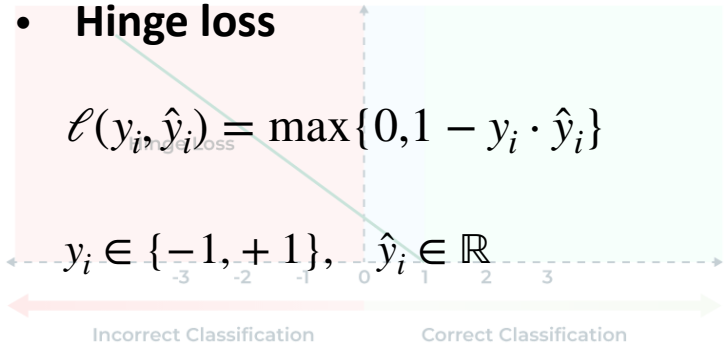
$$\ell(y_i, \hat{y}_i) = \begin{cases} 1, & y_i \neq \hat{y}_i \\ 0, & y_i = \hat{y}_i \end{cases}$$

$$y_i, \hat{y}_i \in \{0,1\} \text{ or } \{-1,+1\}$$

- **Hinge loss**

$$\ell(y_i, \hat{y}_i) = \max\{0, 1 - y_i \cdot \hat{y}_i\}$$

$$y_i \in \{-1,+1\}, \quad \hat{y}_i \in \mathbb{R}$$



- **Perceptron loss**

$$\ell(y_i, \hat{y}_i) = \begin{cases} 1, & |y_i - \hat{y}_i| > t \\ 0, & |y_i - \hat{y}_i| \leq t \end{cases}$$

$$y_i, \hat{y}_i \in \mathbb{R}, \quad t \geq 0$$

- **Squared loss**

$$\ell(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$

$$y_i, \hat{y}_i \in \mathbb{R}$$

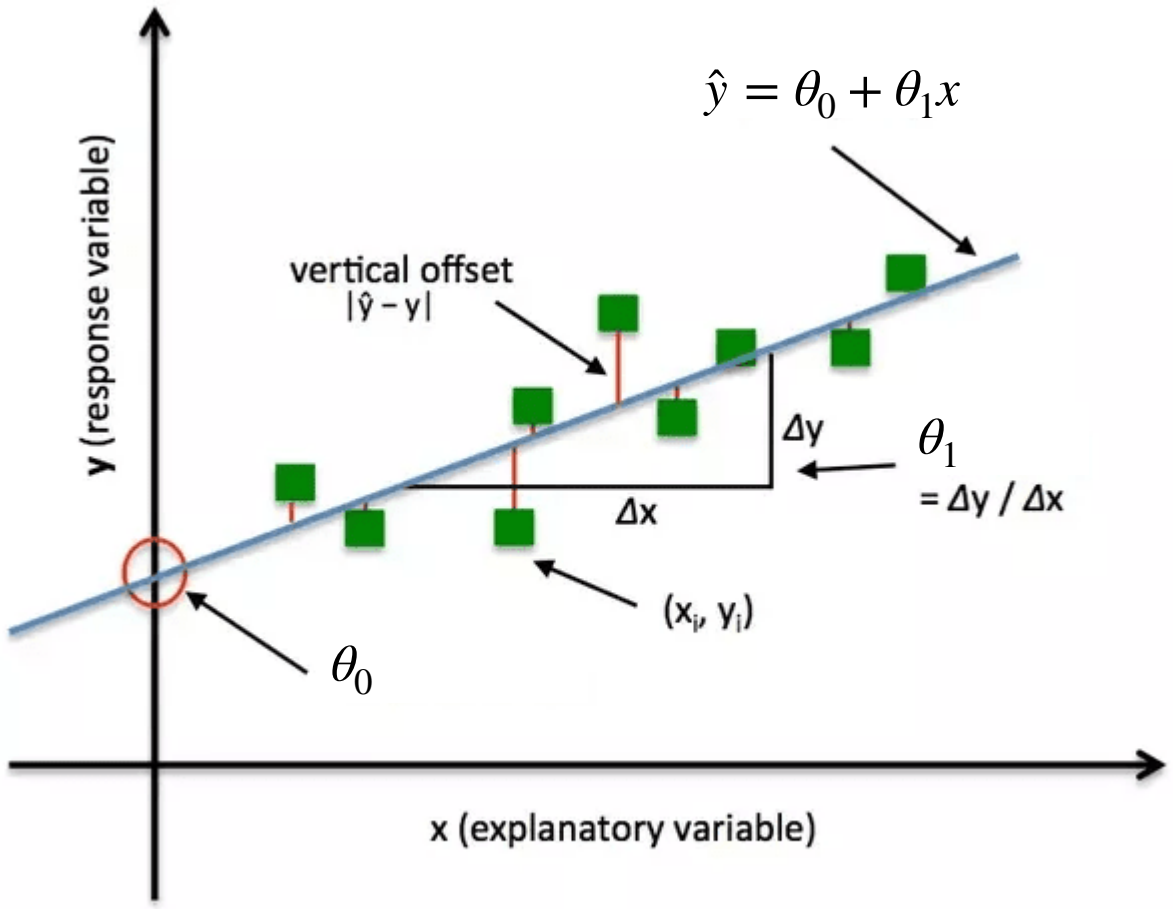
- **Logistic loss**

$$\ell(y_i, \hat{y}_i) = -y_i \log(\hat{y}_i) - (1 - y_i)\log(1 - \hat{y}_i)$$

$$y_i \in \{0,1\}, \quad \hat{y}_i \in (0,1)$$

Evaluation of Machine Learning

Linear Regression: Predicting continuous values by fitting a linear relationship between inputs and targets



There exists different losses for evaluating linear regression performance

L1 loss : $\sum_{i=1}^{n} |y_i - \hat{y}_i|$

Mean Absolute Error (MAE) : $\frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$

L2 loss : $\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$

Mean Squared Error (MSE) : $\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$

https://www.datarobot.com/blog/introduction-to-loss-functions/

Outline

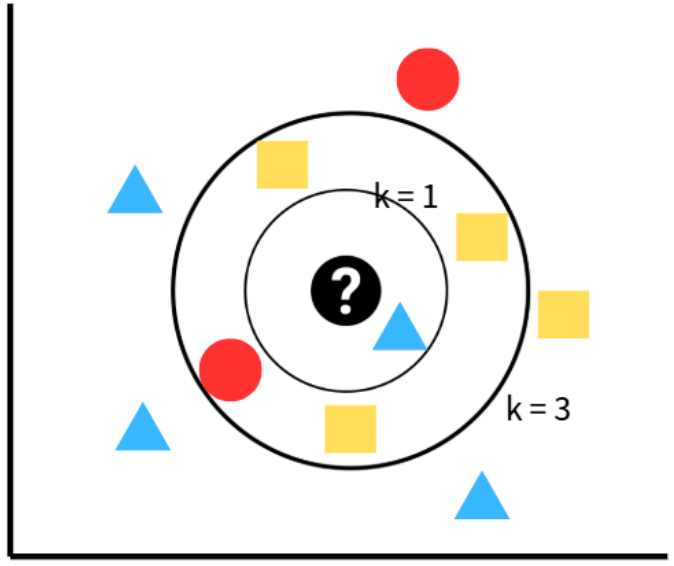
- Class information
- What is Machine Learning (ML)
- ML Problem class
- Assumptions
- Evaluation / Verification
- **ML types and procedures**
- Basic algorithms

Model Type of Machine Learning

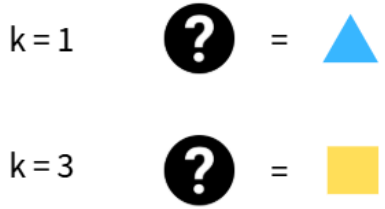
Model-free: guessing directly from the data

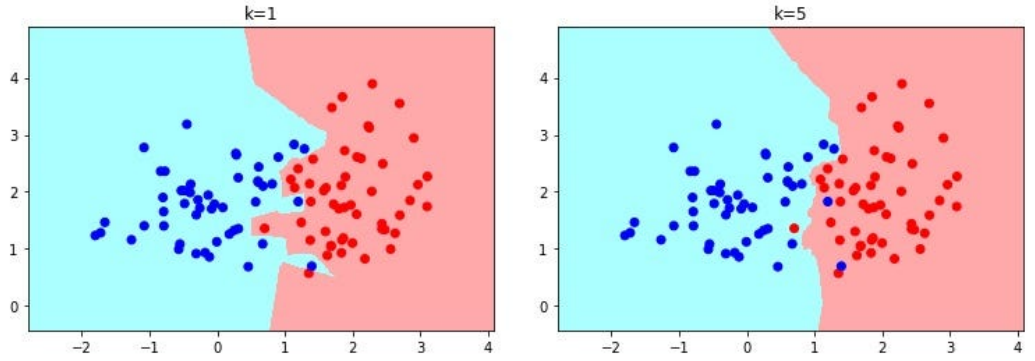
Model-free

- Directly learn from data without explicit model
- Examples: **k-Nearest Neighbors, non-parametric methods**
- Pros: Simple, flexible
- Cons: Requires lots of data, low interpretability



After KNN algorithm :





Model type of Machine Learning

Model-based: use a model to fit data, and guess

Model-based

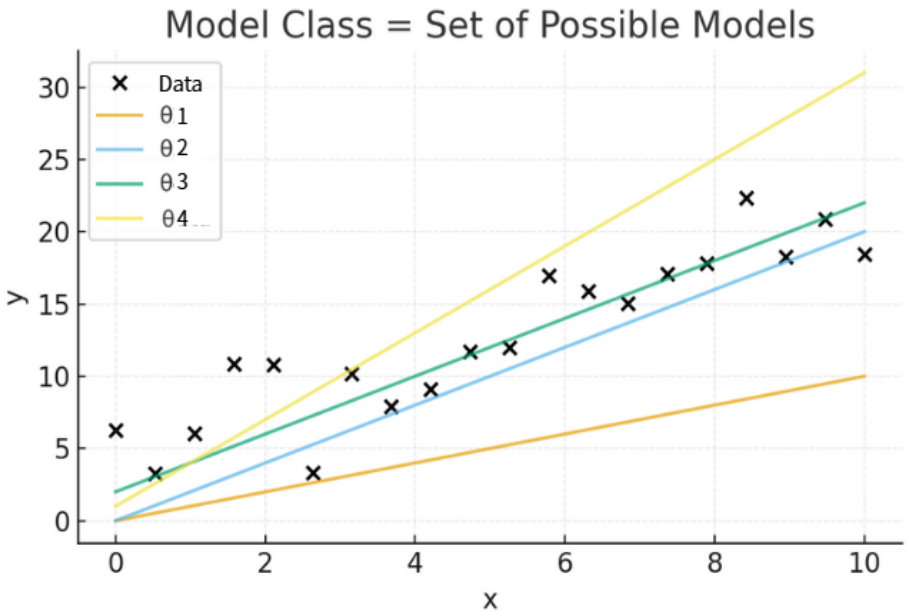
- Assume a model structure, fit parameters to data
- Fitting is usually an **optimization process**
- Examples: **Linear Regression, Logistic Regression, Neural Networks**
- Pros: Interpretable (for simple models), efficient once trained
- Cons: May fail if model assumption is wrong

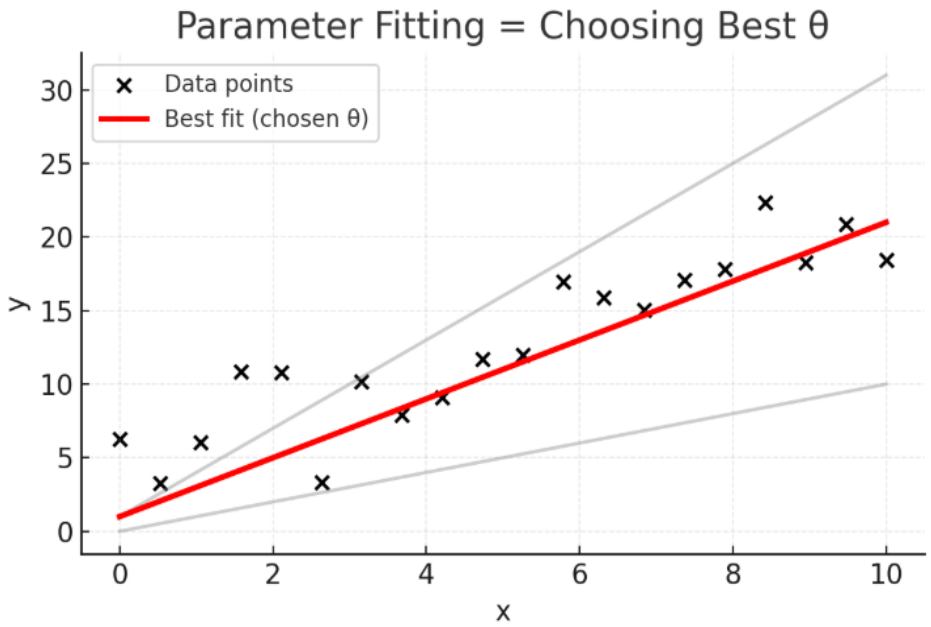
Procedure of Machine Learning

Model class and parameter fitting

- Hypothesis set : a finite set of possible models (hypothesis), parameterized by
- Parameter Fitting : find the best θ that matches the data
- Optimization

Hypothesis set





Algorithm of Machine Learning

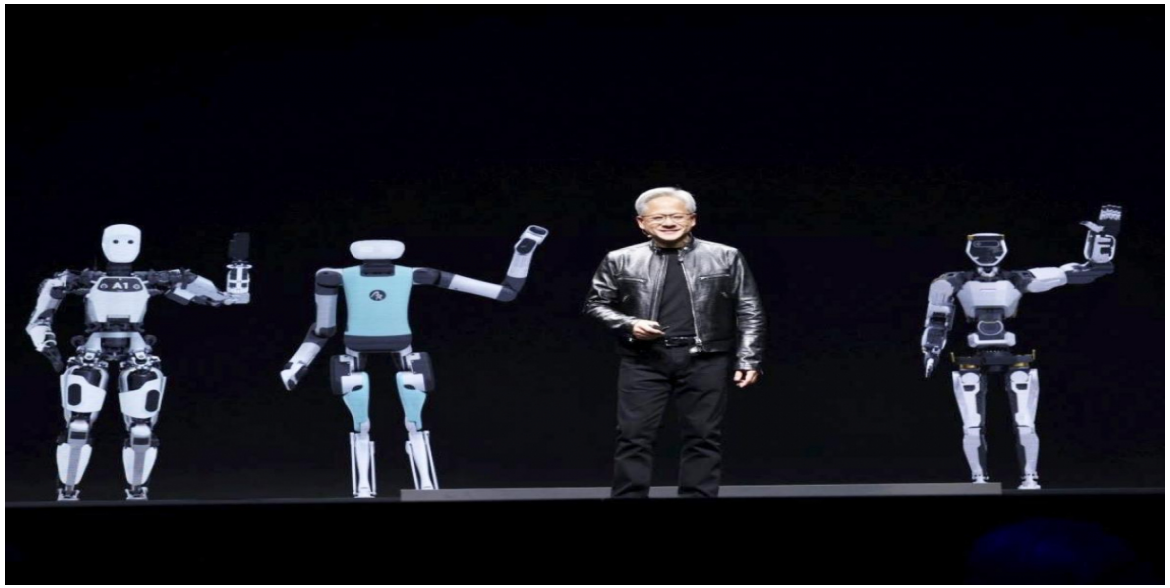
Deciding the best model to use

- We defined a **Hypothesis set** (set of possible hypotheses)
- We have a way of **evaluation metric** for given data (loss / likelihood)
- Now: **How to find a good model?**
- Need a sequence of computational steps = **Algorithm**
- **Example** :

    1. Least-squares minimization → for Linear Regression
    2. Gradient Descent → general optimization
    3. Expectation-Maximization → for latent variable models
    4. Dynamic Programming / Value Iteration → in RL

Application of Machine Learning

天气预报



Outline

- Class information
- What is Machine Learning (ML)
- Problem class
- Assumptions
- Evaluation / Verification
- ML types and procedures
- **Basic algorithms**

Linear Classifier

Basic concept

Difference of classifiers and models

- Classifier (Learning process)
  - An algorithm
  - Learns to assign labels or categories based on input features
- Model (Learning result)
  - The output function after training
  - Used to make prediction

Classification problem -> Assign a category to new data

- Email classification
- Weather prediction
- Image tagging

Training Dataset

- $D_n = \{(x^{(1)}, y^{(1)}), \ldots\ldots, (x^{(n)}, y^{(n)})\}$
- $x^{(i)}$: Input features (d-dimensional vector)
- $y^{(i)}$: Target label (discrete set)

{ Binary classification

Multi-class classification

Linear Classifier

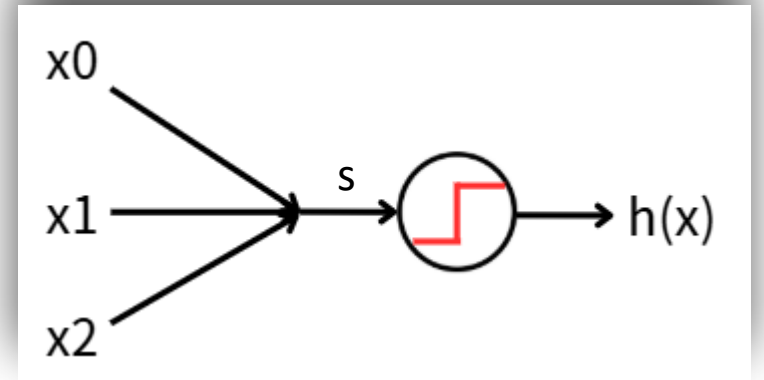
Linear classifier

A fundamental binary classification algorithm that separates data points using a linear decision boundary based on weighted feature combinations

- Linear scoring function :

  $s = \theta^T x + \theta_0 = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d$

- $h(x) = \text{sign}(s) = \text{sign}(\theta^T x + \theta_0)$
- $\text{err}(h, x, y) = I[h(x) \neq y]$



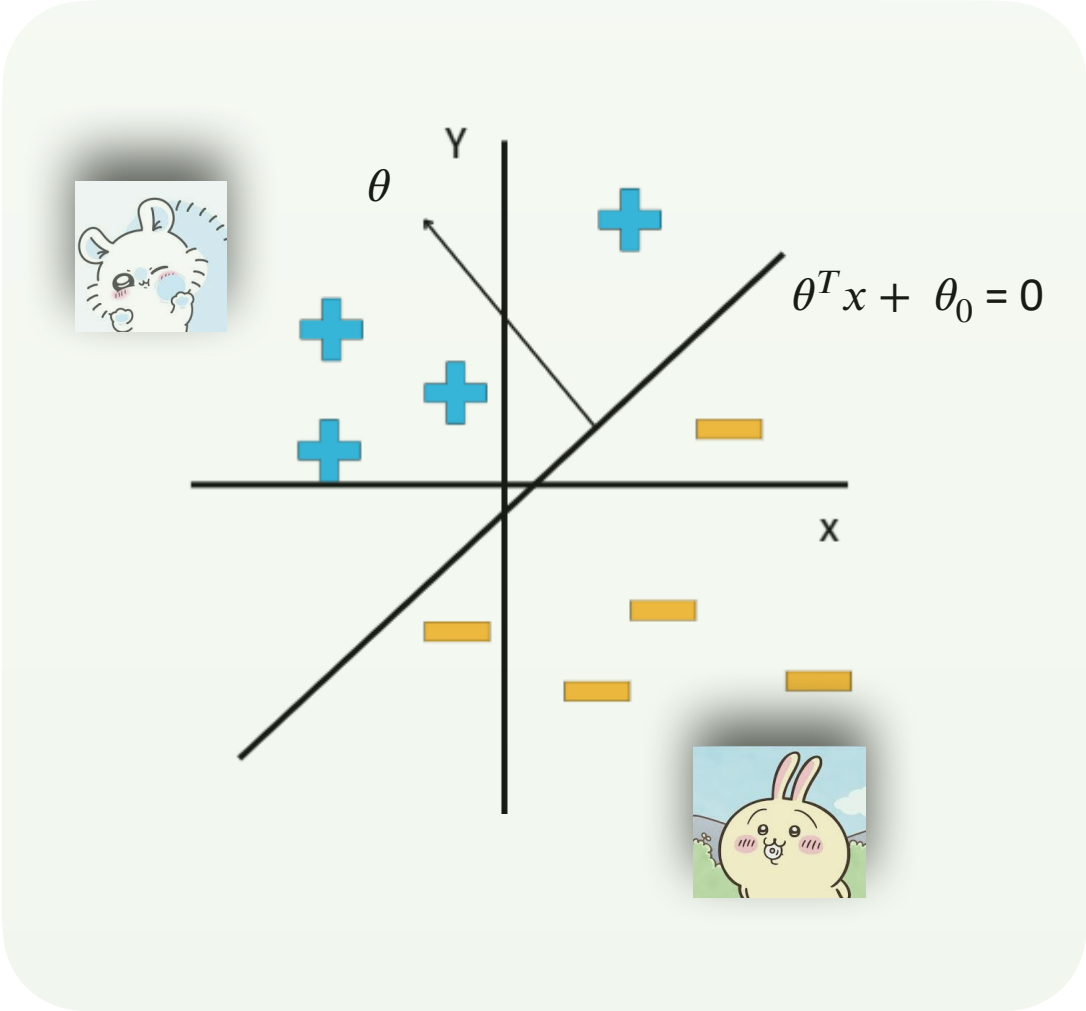
0-1 Loss function:

- $\text{err}_{0/1}(s, y) = I[\text{sign}(s) \neq y]$
- Decision boundary: The hyperplane $\theta^T x + \theta_0$ separates the two classes
- θ is the normal vector to the decision boundary
- $|s|/||\theta||$ gives the distance from point x to the decision boundary

- Linear classifiers learn a hyperplane that optimally divides the feature space into two regions, assigning different class labels to each side
- The decision is made by computing a linear combination of input features and applying a threshold.

Linear Classifier

Random linear classifier



$$h(x;\theta,\ \theta_0) = sign(\theta^T x + \theta_0) = \begin{cases} +1, & if\ \theta^T x + \theta_0 \\ -1, & otherwise \end{cases}$$

$$\mathscr{D}_n = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

d = number of features/dimensions in input space

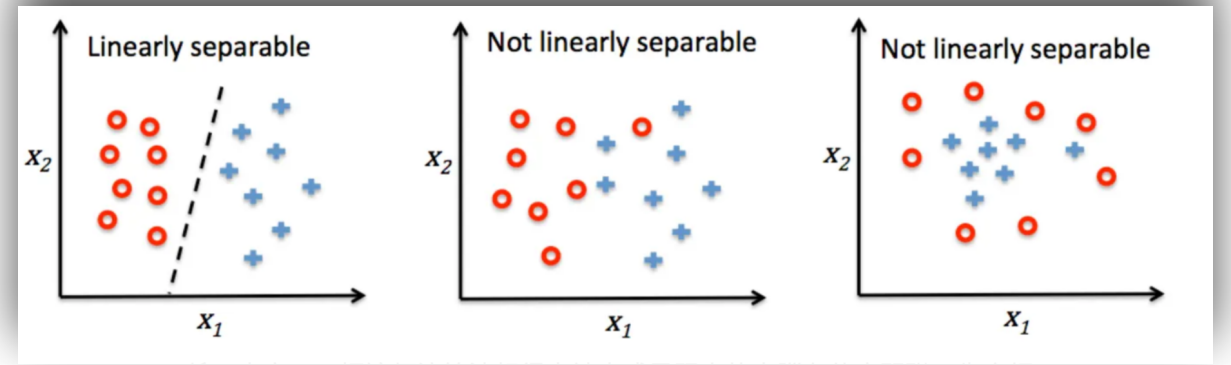
$$\epsilon_n(\theta, \theta_0) = \frac{1}{n}\sum_{i=1}^{n} I[y_i \neq \text{sign}(\theta^T x_i + \theta_0)]$$

RANDOM-LINEAR-CLASSIFIER($\mathscr{D}_n, k, d$)

1. For $j = 1$ to k
2. Randomly sample $(\theta^{(j)}, \theta_0^{(j)})$ from $(\mathbb{R}^d, \mathbb{R})$
3. $j^* = \arg\min_{j \in \{1,\ldots,k\}} \epsilon_n(\theta^{(j)}, \theta_0^{(j)})$
4. return $(\theta^{(j^*)}, \theta_0^{(j^*)})$

Perceptron Learning Algorithm (PLA)

Basic machine learning algorithm

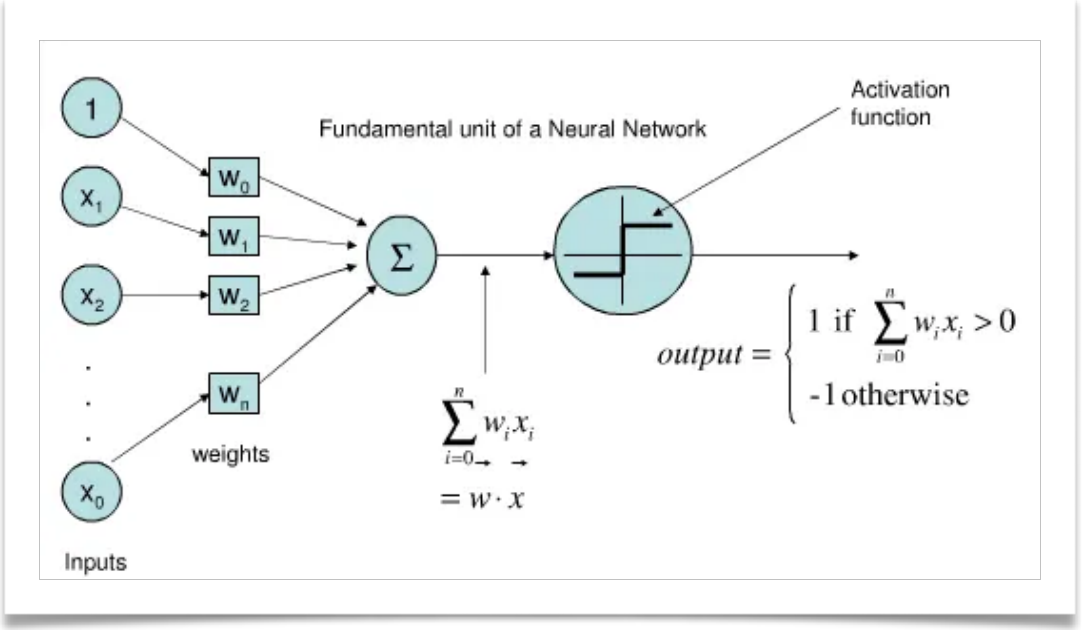


Perceptron

- Iterative weight update rule
- Works only with linearly separable data

Convergence theorem

The basic result about the perceptron is that, if the training data is **linearly separable**, then the perceptron algorithm is guaranteed to find a linear separator



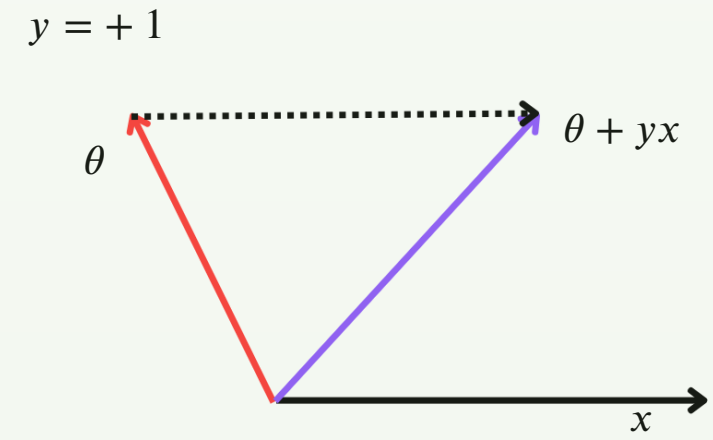
https://medium.com/jameslearningnote/%E8%B3%87%E6%96%99%E5%88%86%E6%9E%90-%E6%A9%9F%E5%99%A8%E5%AD%B8%E7%BF%92-%E7%AC%AC3-2%E8%AC%9B-%E7%B7%9A%E6%80%A7%E5%88%86%E9%A1%9E-%E6%84%9F%E7%9F%A5%E5%99%A8-perceptron-%E4%BB%8B%E7%B4%B9-84d8b809f866

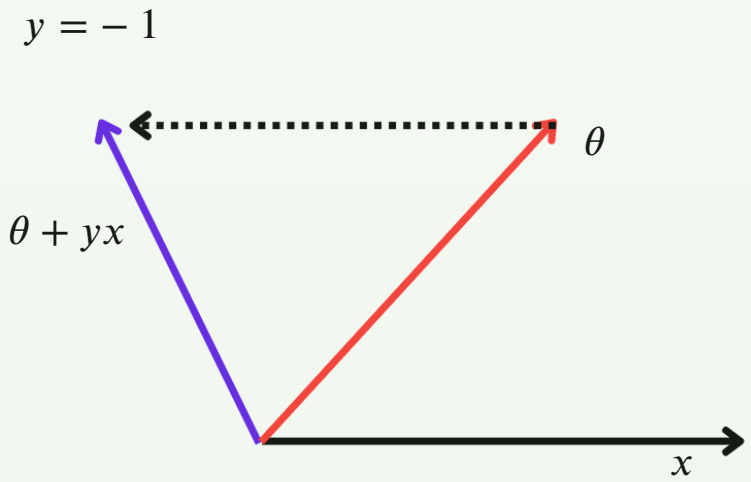
Perceptron Learning

Basic machine learning algorithm

An iterative algorithm that learns linear decision boundaries by correcting misclassification errors

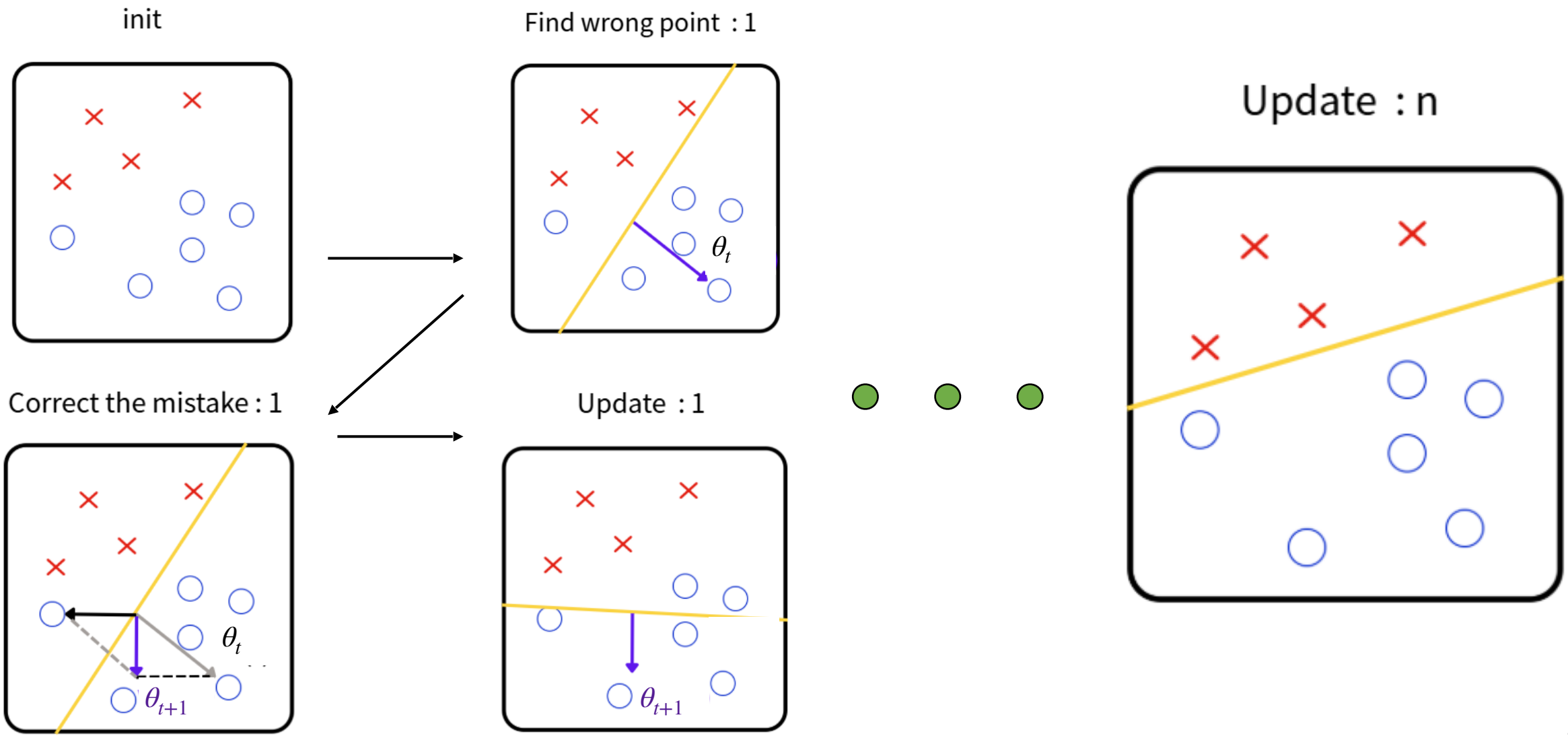
1. **Find a mistake**
   - Misclassified point: $(x_{n(t)}, y_{n(t)})$
   - Condition: $\text{sign}(\theta_t^T x_{n(t)}) \neq y_{n(t)}$
2. **Correct the mistake**
   - $\theta_{t+1} = \theta_t + y_{n(t)} x_{n(t)}$
3. **Repeat until:**
   - No mistakes occur, or
   - A stopping condition is reached (e.g. max iterations)





Perceptron Learning

Procedure



Linear Regression

Find the relationship between input features and a continuous output

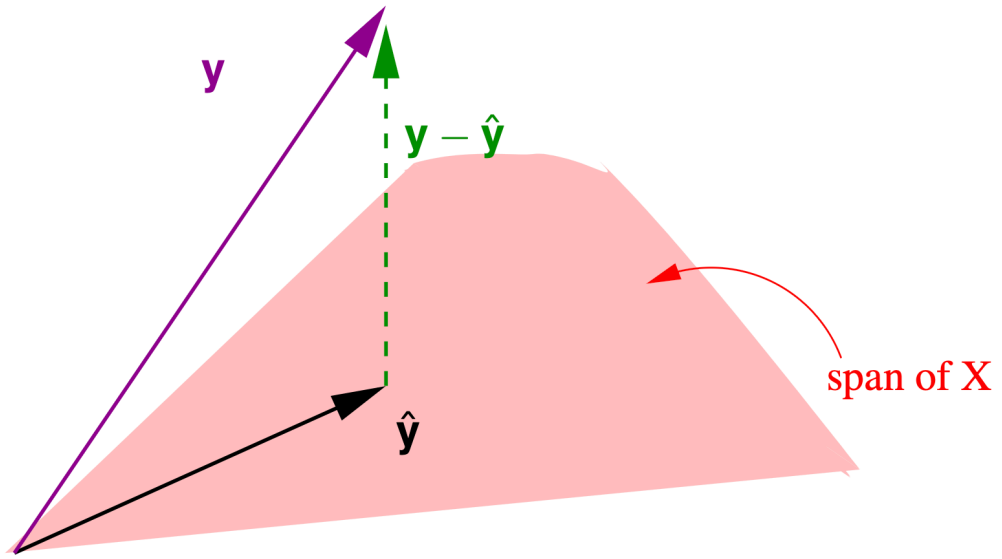
---

Objective function

$$J(\theta) = \frac{1}{2N}\sum_{n=1}^{N}(h_\theta(x^{(n)}) - y^{(n)})^2 = \frac{1}{N}\sum_{n=1}^{N} Loss(h_\theta(x^{(n)}), y^{(n)})$$

Loss function

$$Loss(h_\theta(x), y) = \frac{1}{2}(h_\theta(x) - y)^2$$

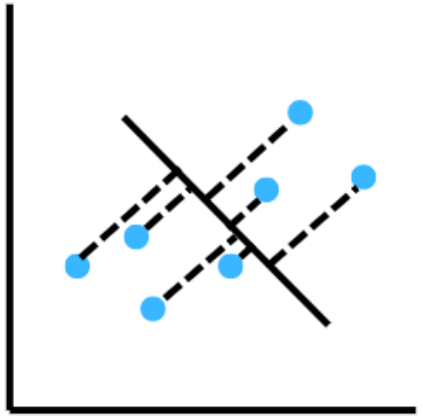


Convexity

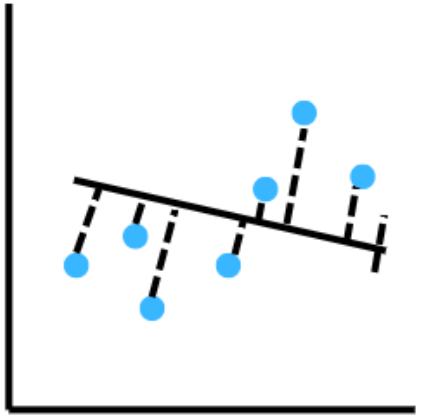
- Square loss is convex.
- Guarantees a unique global optimum.

Linear Regression

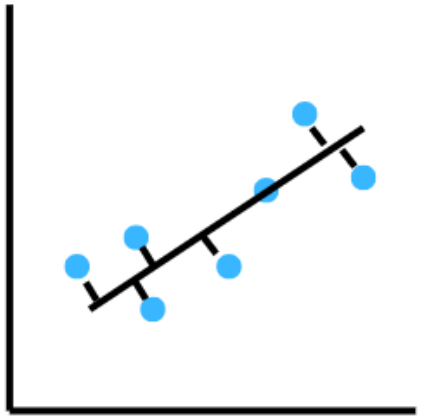
Find the relationship between input features and a continuous output



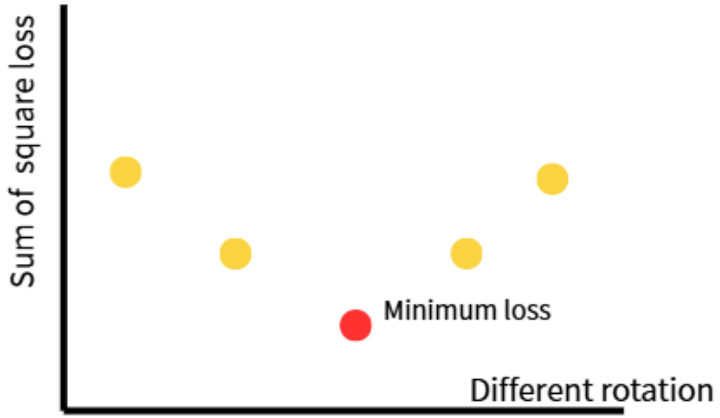
High Loss



High Loss



Low Loss



Linear Regression

Find the relationship between input features and a continuous output

Analytical Solution

- Since the loss function is convex → global minimum is found when gradient = 0
- Derivation (simplified) :

$$L(\theta) = (Y - X\theta)^T(Y - X\theta)$$
$$\nabla_\theta L(\theta) = -2X^T(Y - X\theta) = 0$$

- Leads to Normal Equation:

$$\theta = \left(X^T X\right)^{-1} X^T Y$$

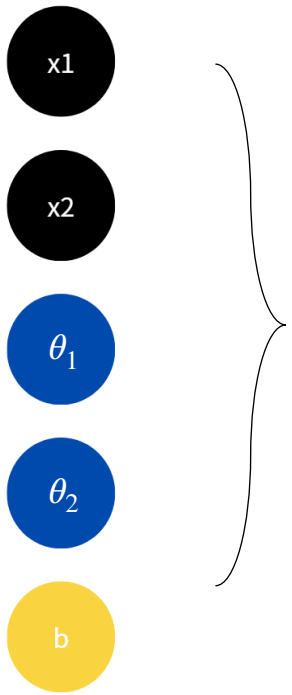
Pseudo-Inverse

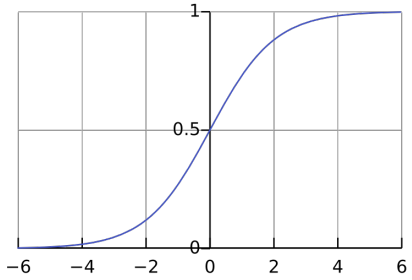
- Problem: $X^T X$ may be **non-invertible**
- Solution: use Moore - Penrose pseudo-inverse → $\theta = X^\dagger Y$
- Guarantees a valid least-squares solution even when $X^T X$ is singular.
- Ensures for computing the **best-fit line**

Logistic Regression

Probability estimation

- Logistic regression predicts probability
- Uses **sigmoid function** to map inputs into range (0,1)





Sigmoid function

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

$$z = x_1\theta_1 + x_2\theta_2 + b \quad \rightarrow \quad \hat{y} = \sigma(z) \quad \rightarrow \quad \mathcal{L}(\hat{y}, y)$$

$$\mathcal{L}\left(\hat{y}, y\right) = -\left(y \log \hat{y} + \left(1 - y\right)\log(1 - \hat{y})\right)$$

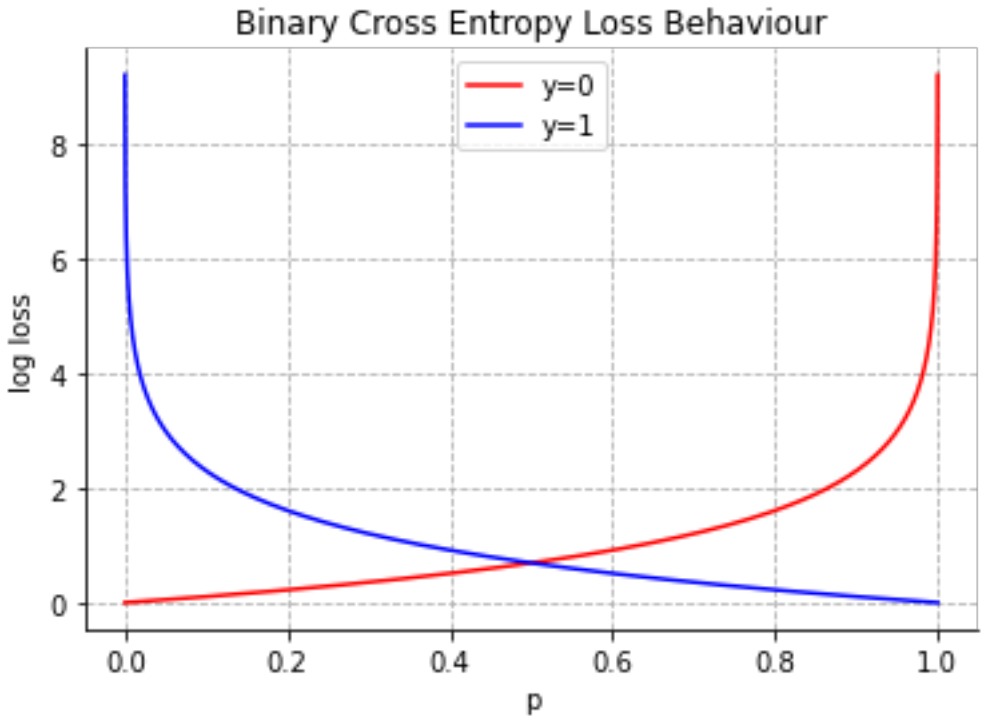
Cross-Entropy Loss

Logistic Regression

Error Measure: Cross-Entropy

- Likelihood maximization = fitting parameters to maximize probability of observed data
- Binary logistic regression likelihood
  - $P(y \mid \hat{y}) = \hat{y}^{y}(1-\hat{y})^{1-y}$
- Log-likelihood
  - $\log P(y \mid \hat{y}) = y \log(\hat{y}) + (1-y)\log(1-\hat{y})$
- Loss function

$$\mathscr{L}(\hat{y}, y) = -\log P(y \mid \hat{y}) = -\left(y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right)$$



https://www.desmos.com/calculator/zytm2sf56e

https://www.desmos.com/calculator/zytm2sf56e

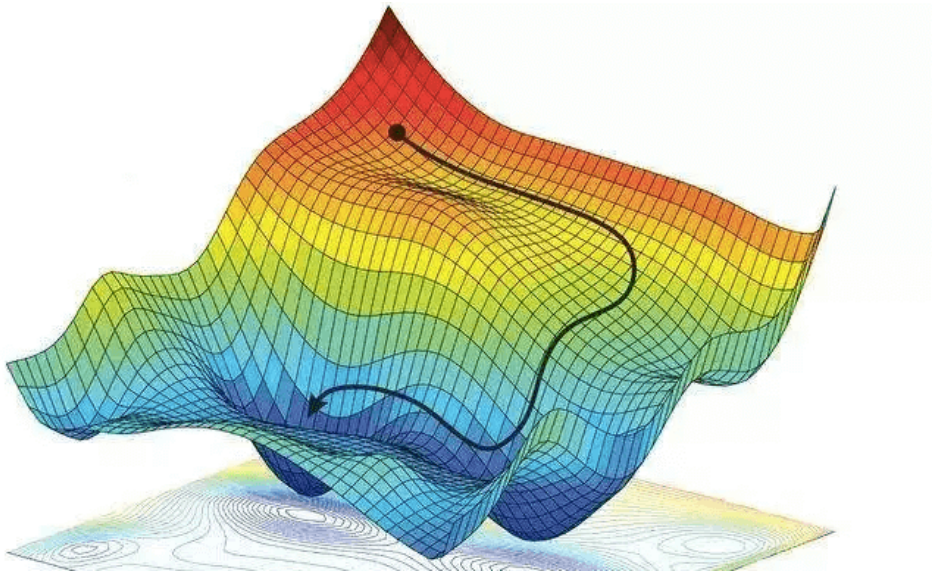
Logistic Regression

Iterative Optimization: Gradient descent

- No closed-form solution → must use gradient descent
- Gradient descent (local approximation by linear formula)
- Update rule:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta L(\theta), \qquad \eta \; is \; learning \; rate$$

- Gradient descent

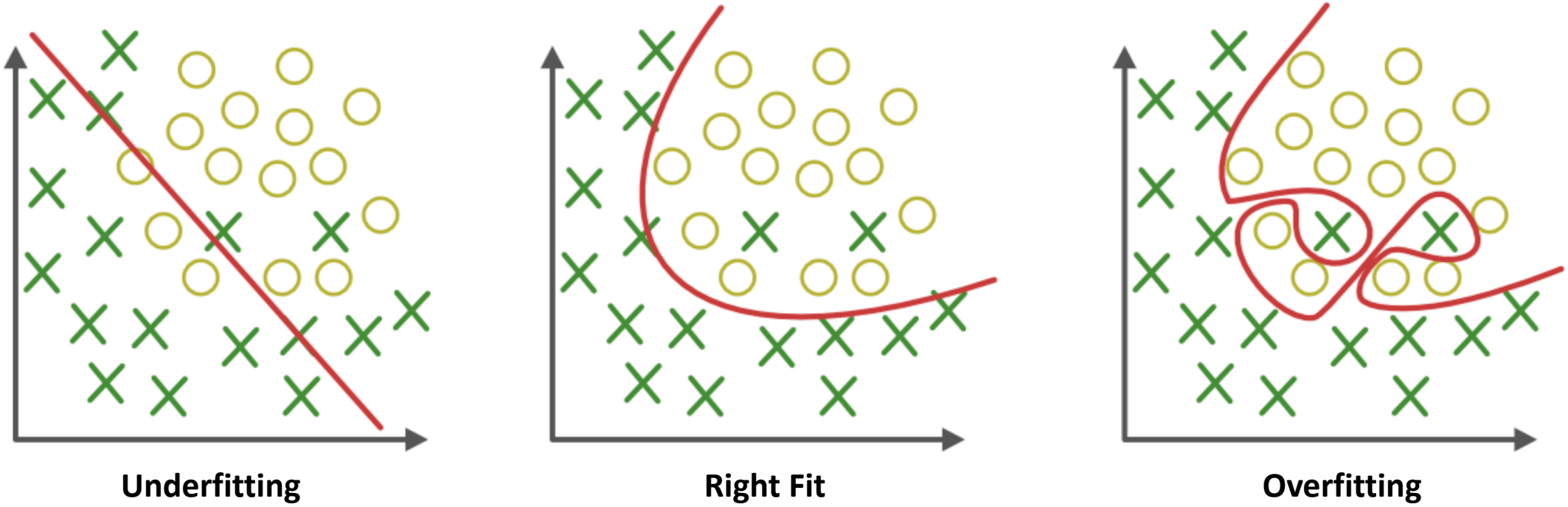


https://mriquestions.com/back-propagation.html

Overfitting

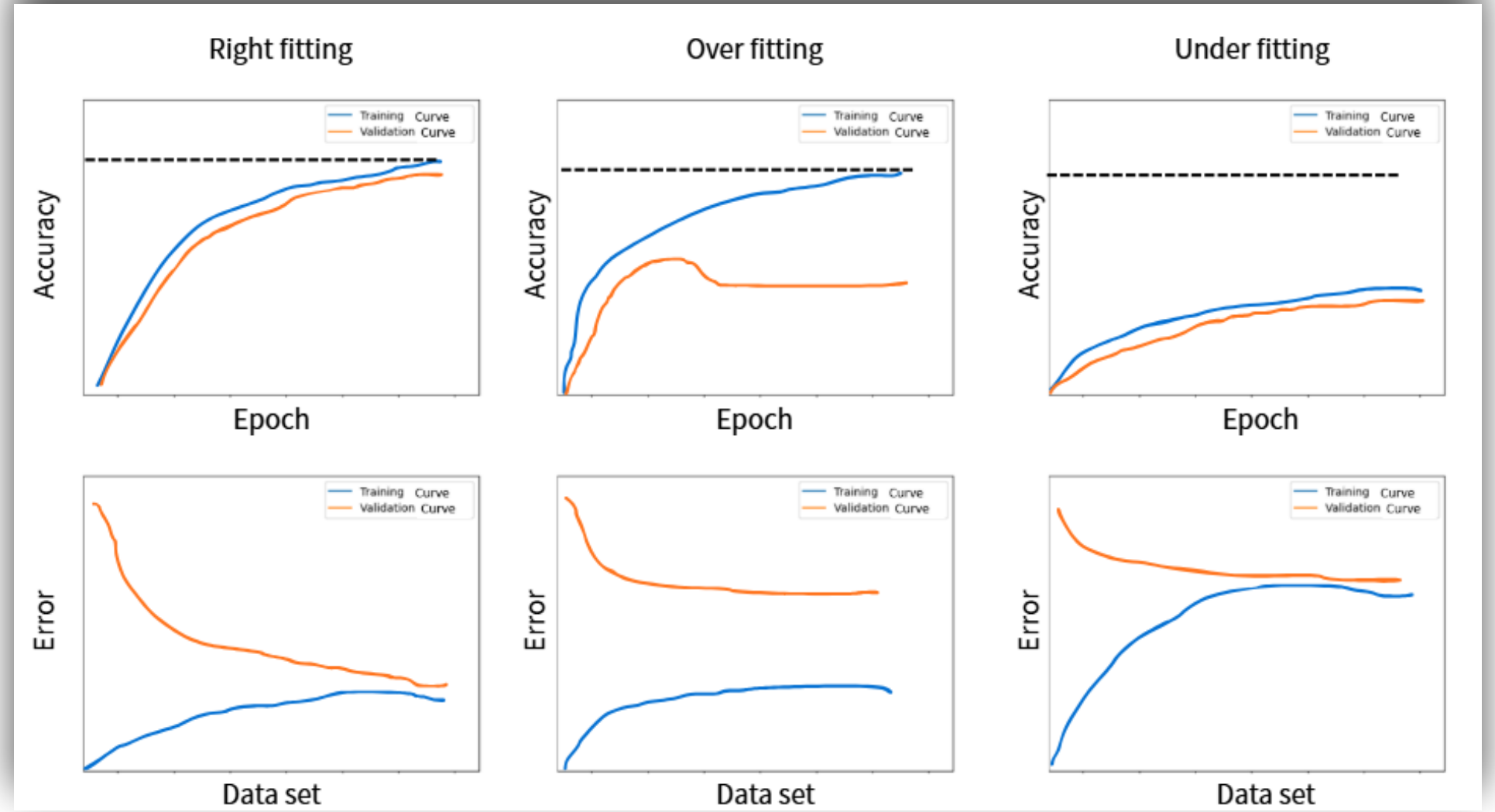
Model fits too closely to the training data

- Linear models are sometimes **too weak** → cannot capture complexity
- Add quadratic / polynomial terms or non-linear transforms
- Model becomes too complex → generalization suffers
- **Curse of dimensionality**: higher dimensions need more data, otherwise risk of overfitting



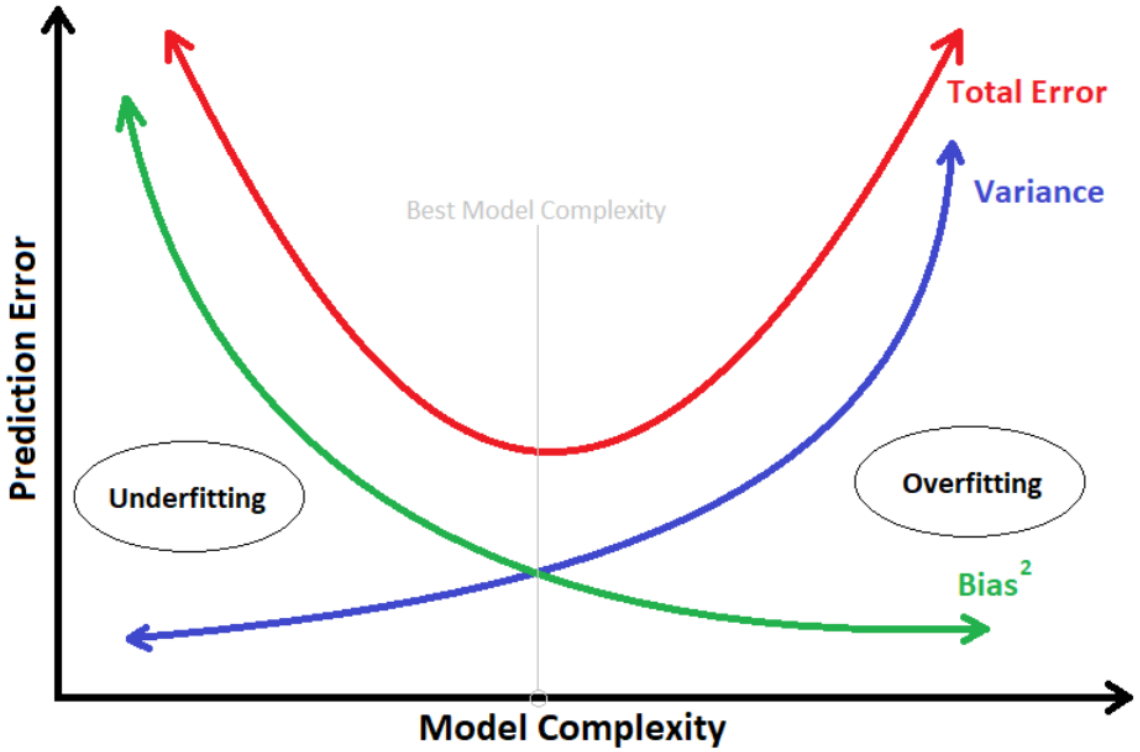
Overfitting

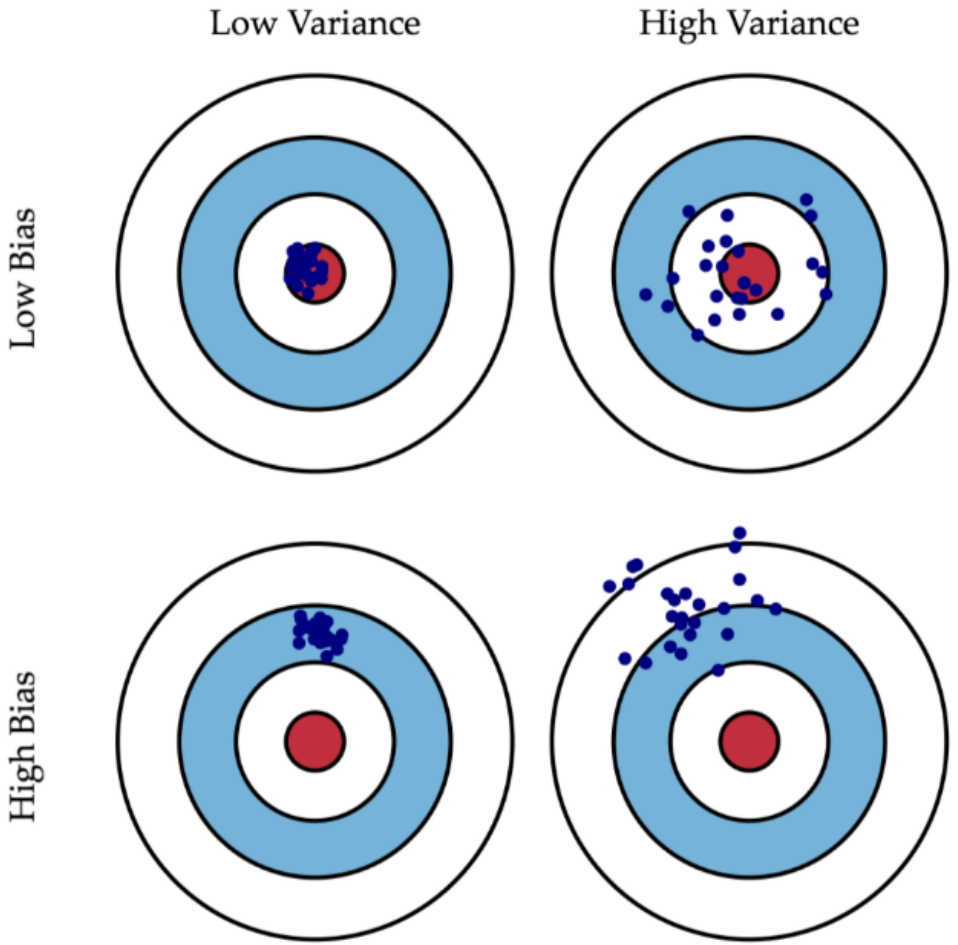
Learning curve introduction



Bias-Variance Trade-Off

Find the sweetest spot for best generalization

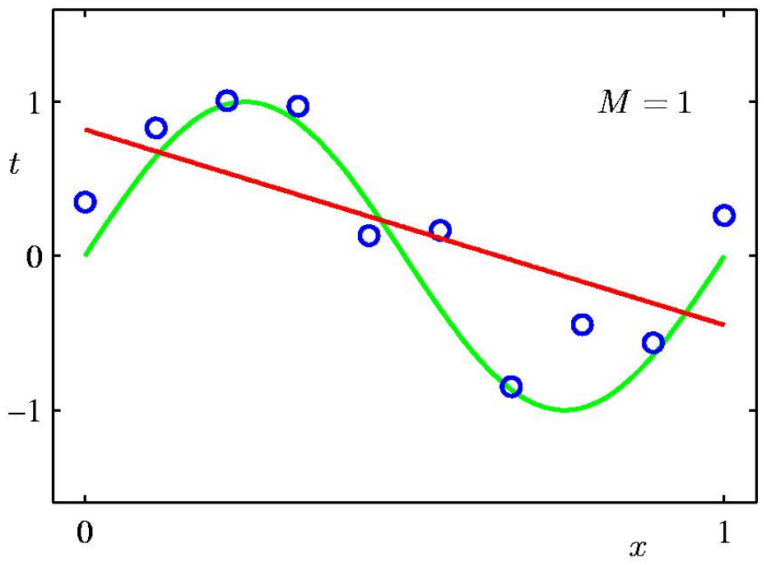




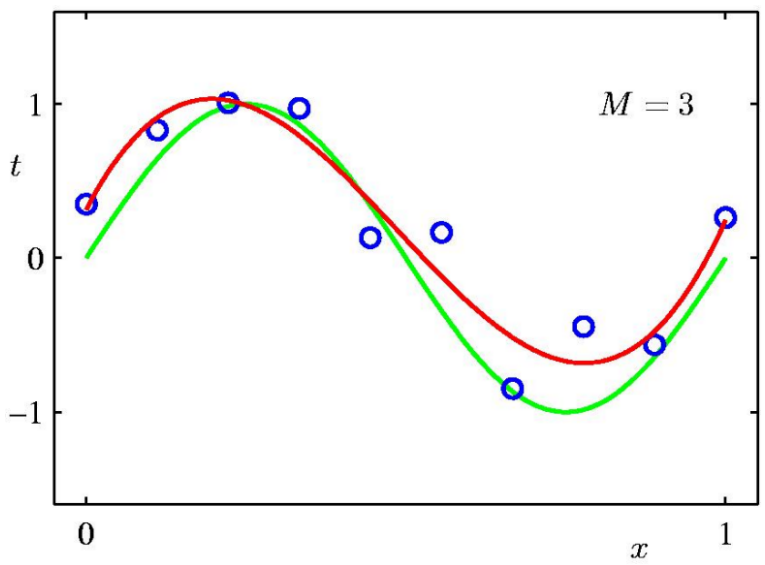
Regularization

Simplifying the model

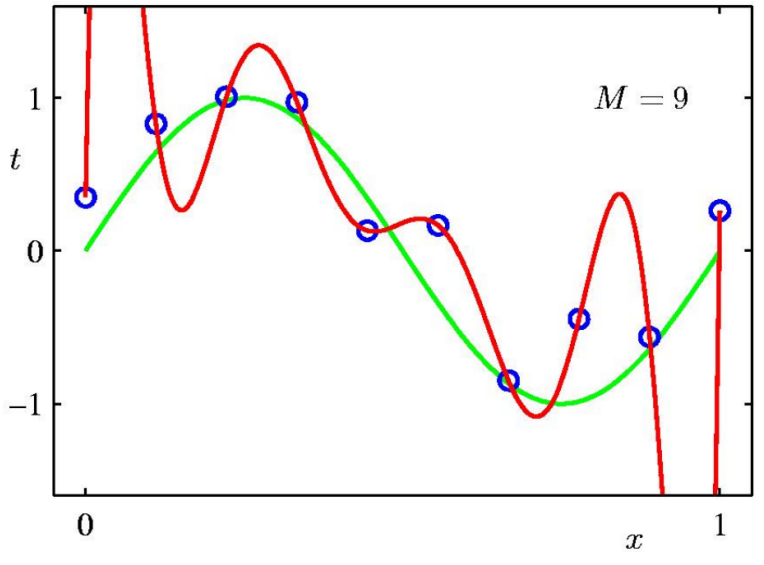
- Complex models → too many parameters → risk of overfitting
- Need to **control hypothesis space** → regularization



1st Order Polynomial



3rd Order Polynomial



9th Order Polynomial

Regularization

Simplifying the model

- (1) L1 norm (Lasso) $$||\theta||_1 = \sum_{j=1}^{m} |\theta_j|$$

(2) L2 norm (Ridge) $$||\theta||_2^2 = \sum_{j=1}^{m} \theta_j^2$$

- **Lagrange Multiplier Derivation**

$$\min_{\theta} \sum_{i=1}^{n} \mathscr{L}\left(f(x_i), y_i\right) \quad \text{s.t.} \quad \|\theta\|_p \le C$$

$$\min_{\theta} \sum_{i=1}^{n} [\mathscr{L}\left(f(x_i), y_i\right) + \lambda\left(\|\theta\|_p - C\right)]$$

$$\min_{\theta} \sum_{i=1}^{n} [\mathscr{L}\left(f(x_i), y_i\right) + \lambda\|\theta\|_p]$$

L1 Regularization

$$\min_{\theta} \sum_{i=1}^{n} \mathscr{L}\left(f(x_i), y_i\right) + \lambda \sum_{j=1}^{p} \left|\theta_j\right|$$

L2 Regularization

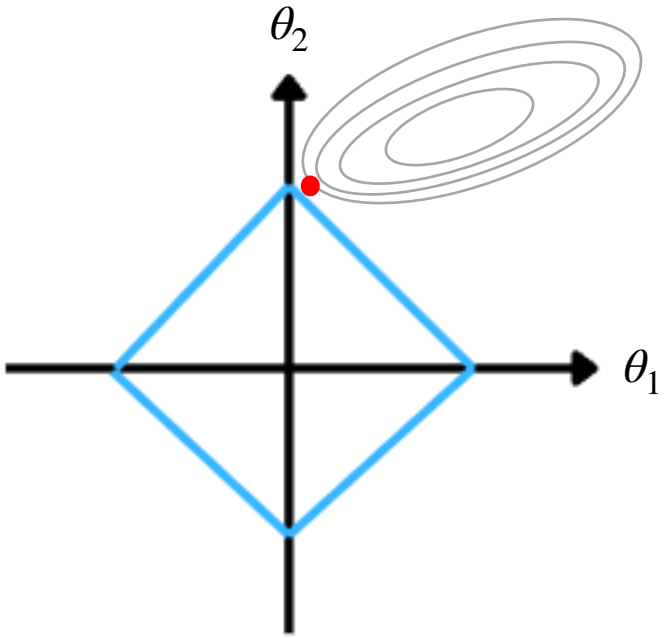
$$\min_{\theta} \sum_{i=1}^{n} \mathscr{L}\left(f(x_i), y_i\right) + \lambda \sum_{j=1}^{p} \theta_j^2$$

Elastic Net Regularization

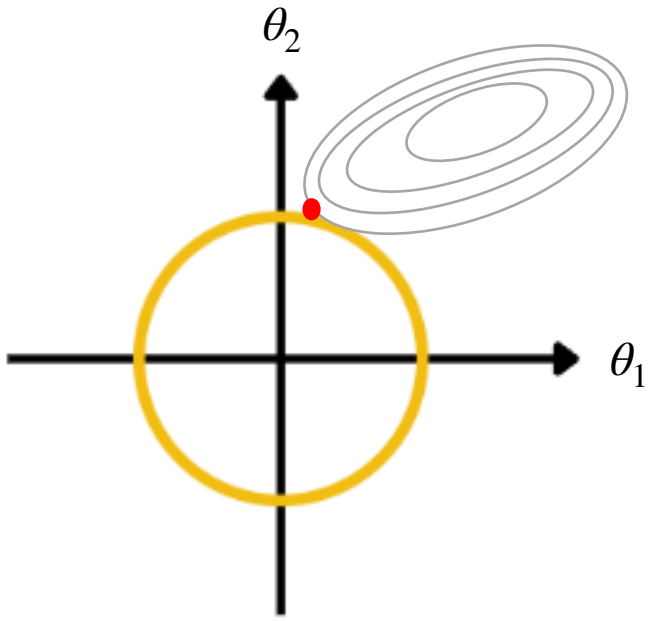
$$\min_{\theta} \sum_{i=1}^{n} \mathscr{L}\left(f(x_i), y_i\right) + \lambda_1 \sum_{j=1}^{p} \left|\theta_j\right| + \lambda_2 \sum_{j=1}^{p} \theta_j^2$$

Regularization

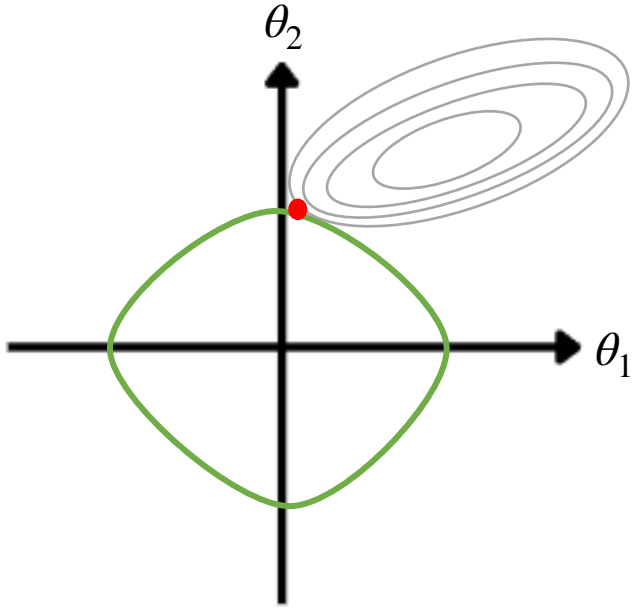
Visualization



L1 Regularization



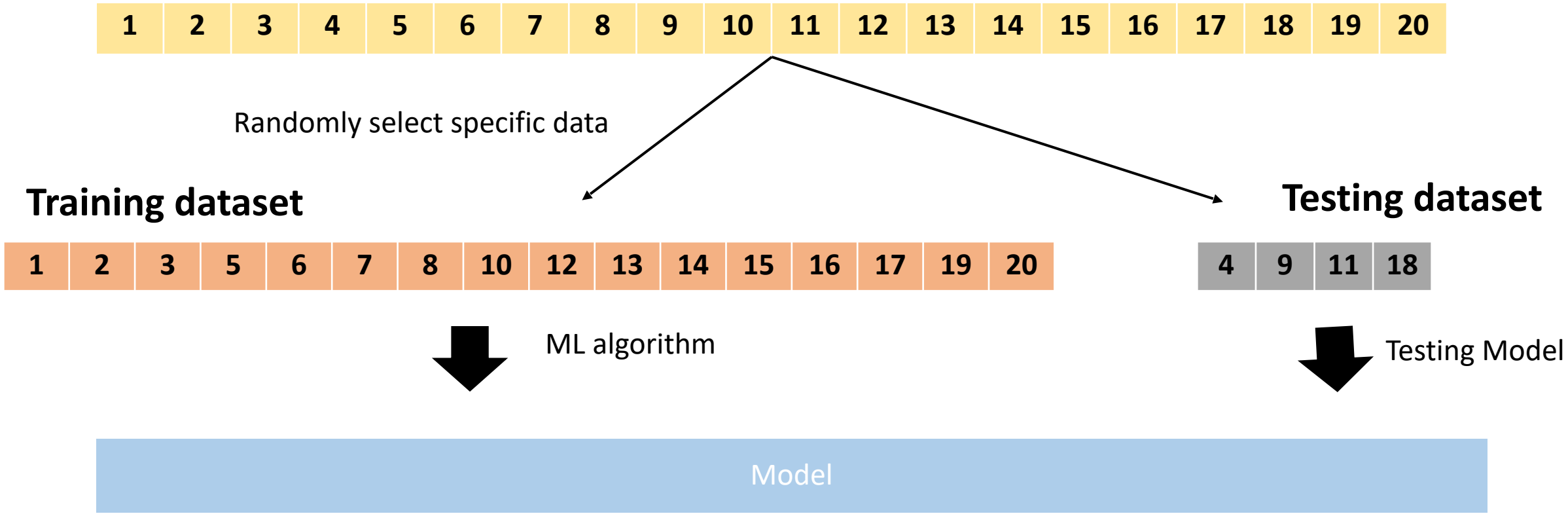
L2 Regularization



Elastic Net Regularization

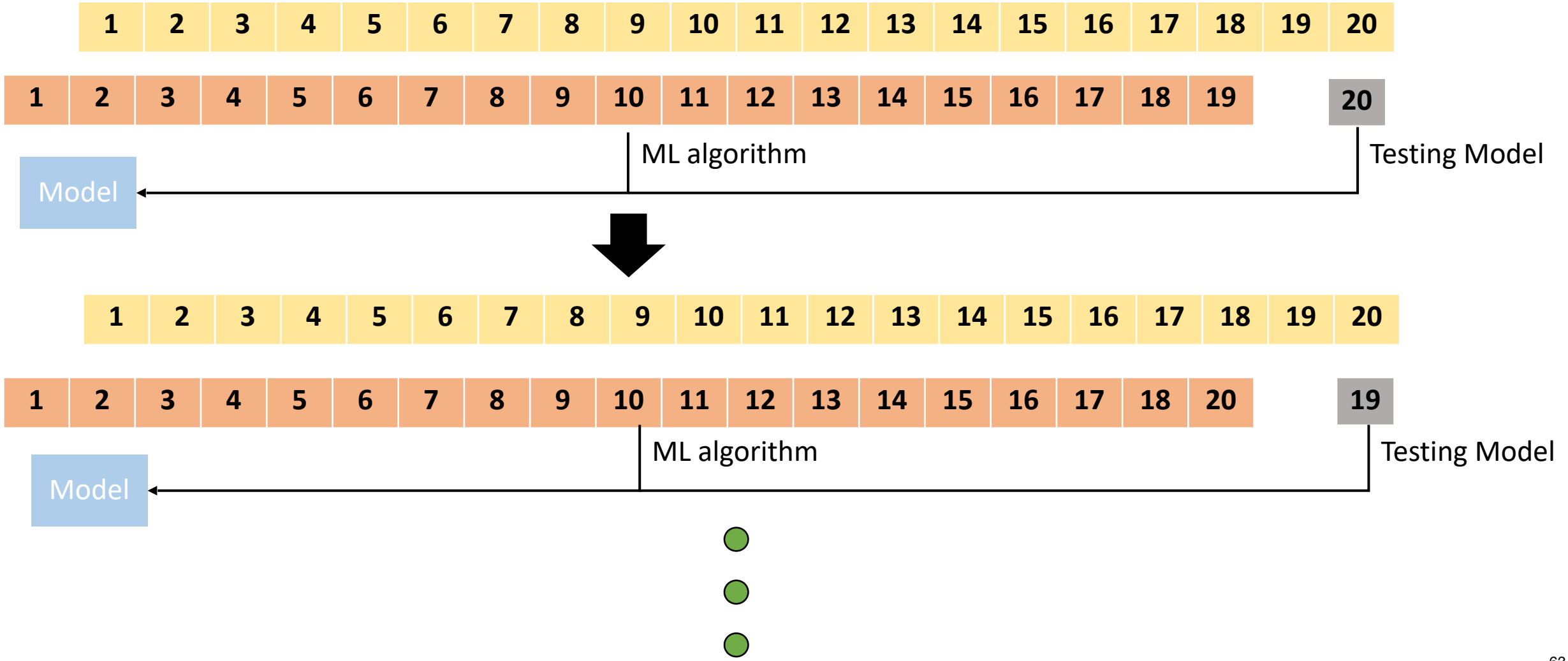
Cross Validation

Holdout Cross-Validation



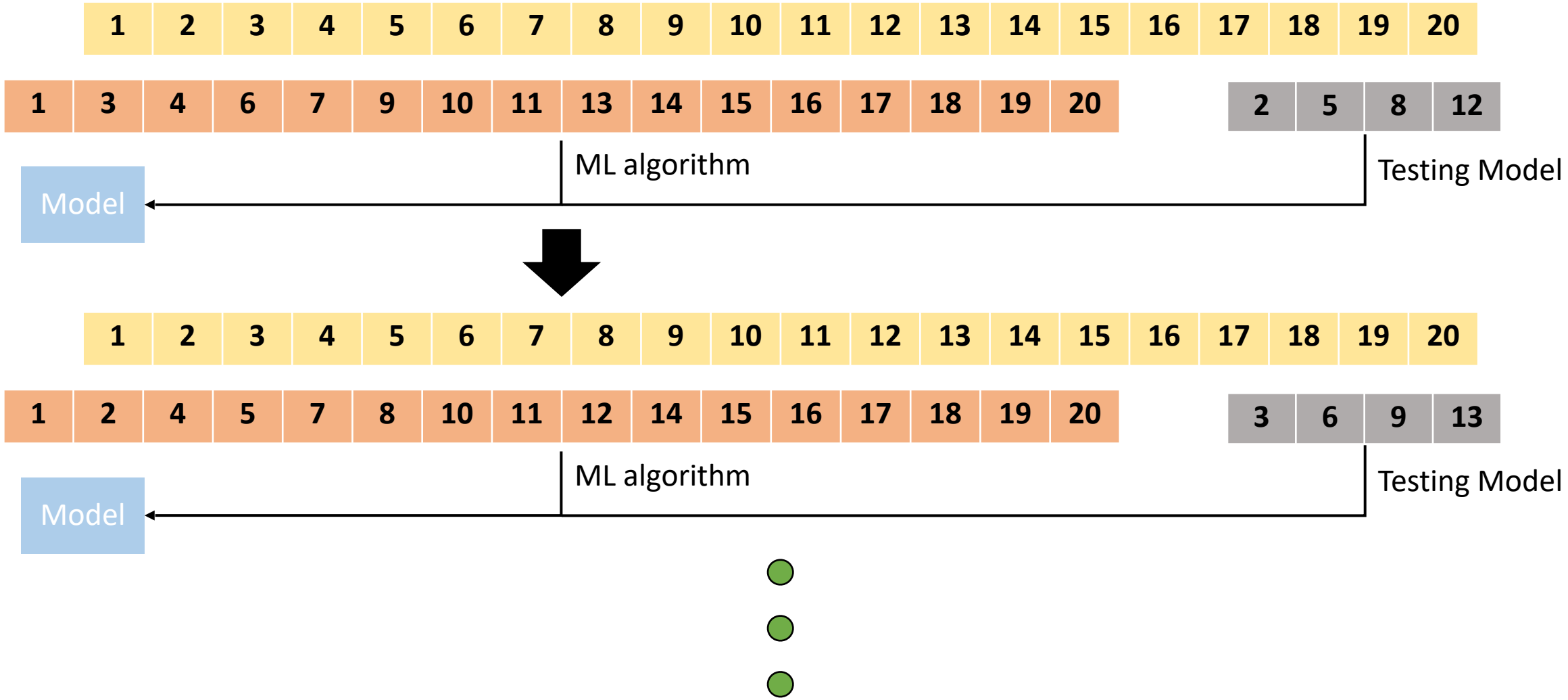
Cross Validation

Leave-One-Out Cross-Validation



Cross Validation

K-Fold Cross-Validation



Thank You for Your Attention!

Scan this if you are interested!